

Contraction-Aware Reinforcement Learning for Nonlinear Control with Statistical Robustness

Minjae Cho, Hiroyasu Tsukamoto, and Huy T. Tran

Abstract—Control contraction metrics (CCMs)—defined by Riemannian metrics under which a closed-loop system is incrementally exponentially stable—offer a constructive framework for synthesizing contracting policies in nonlinear path-tracking problems. However, while the synthesized policies ensure pointwise satisfaction of the CCM conditions, they may not ensure long-term optimality (i.e., minimizing cumulative trajectory-level tracking error) over both transient and steady-state regimes. Furthermore, the myopic nature of these policies could also make them more susceptible to learning biases when approximate dynamics are used to formulate CCMs. To address these issues, we propose to integrate CCMs into reinforcement learning (RL). CCMs provide dynamics-informed feedback for learning a policy that has a stability guarantee—i.e., is contraction-aware—while RL provides a framework for minimizing cumulative tracking error under approximate dynamics. Given a pretrained dynamics model, our algorithm, contraction-aware RL (CARL), simultaneously learns to generate CCMs and optimize a policy for rewards defined by those CCMs. We demonstrate that CARL enhances path-tracking performance and is robust to errors in approximated dynamics compared to relevant baselines in both simulated and real-world robot experiments. We also provide theoretical rationale for integrating CCMs into RL. Our code is available at <https://github.com/Mgineer117/CARL>, and a video of our real-world robot experiments can be found at <https://youtu.be/sOJ4hulbop0>.

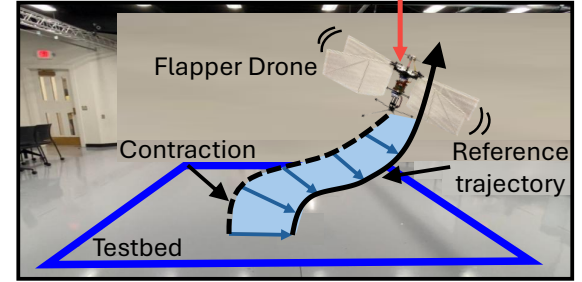
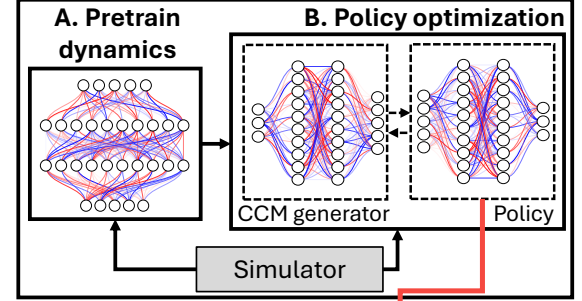
Index Terms—Contraction theory, control contraction metric, reinforcement learning, nonlinear control

I. INTRODUCTION

A fundamental challenge in designing path-tracking control policies, particularly in safety-critical nonlinear systems, is to ensure their reliable operation while efficiently achieving task objectives. Control contraction metrics (CCMs) [1], grounded in contraction theory [2], provide necessary and sufficient conditions for certifying incremental and exponential stabilizability through a path-tracking policy (i.e., a contracting policy) with respect to an arbitrary feasible reference trajectory. This certificate is contingent upon the existence of CCMs, Riemannian metrics under which the closed-loop system is guaranteed to achieve contraction. The ability to provide such a certificate has motivated various methods to synthesize CCMs and corresponding contracting policies for several classes of nonlinear systems.

One approach first identifies CCMs by solving an infinite-dimensional convex feasibility problem using techniques such as sum-of-squares (SoS) programming [1], [3], finite difference methods [4], [5], or reproducing kernel Hilbert space theory for partially known dynamics [6]. These metrics can be

1. Simulation Training



2. Policy Deployment

Fig. 1: Our approach uses simulation-based training, where we first pretrain an approximate dynamics model for robots, which is then used to jointly learn a CCM generator and optimal policy. This policy is then deployed on the real robots for contraction-aware and robust path-tracking.

optimized further, e.g., by minimizing an upper bound on the steady-state tracking error subject to the CCM constraints [3], [4]. The identified CCMs can then be used to synthesize a differential feedback control policy, whose path-integrated form guarantees contraction. However, these methods scale poorly to high-dimensional nonlinear systems as they require extensive computation to derive controls (e.g., solving the feasibility problem for CCMs and finding minimizing geodesics defined by CCMs) [7]. They may also rely on restrictive assumptions on the structure of the system dynamics (e.g., polynomial dynamics).

Sun et al. [8] adopt a learning-based approach to identify CCMs and synthesize contracting policies, thereby addressing the scalability and structural limitations of non-learning methods. They jointly train two neural networks—one for a CCM generator (CMG) and another for a policy—by leveraging violations of CCM conditions as their training loss. However, this approach only ensures the pointwise satisfaction of CCM conditions and therefore does not consider long-term optimality (i.e., cumulative tracking error over an entire trajectory). Furthermore, Richards et al. [9] show that such a myopic formulation could make the synthesized policy more

The authors are with Department of Aerospace Engineering, The Grainger College of Engineering, University of Illinois Urbana-Champaign, Urbana, IL 61801 USA. (e-mail: {minjae5, hiroyasu, huytran1}@illinois.edu)

susceptible to learning biases when approximate dynamics are used to model CCM conditions.

[10] and [11] integrate reinforcement learning (RL) with contraction theory to provide a contraction certificate to RL-trained policies. [10] introduces a coordinate transformation to linearize nonlinear contraction constraints and apply these constraints to the neural network weights of the policy, and [11] introduces an auxiliary reward to the path-tracking reward to encourage satisfaction of the contraction condition. However, the former does not account for long-term optimality or guarantee the existence of such a coordinate transformation, particularly in high-dimensional nonlinear systems, and the latter does not establish that the proposed reward function leads to the actual synthesis of a contracting policy.

In this paper, we integrate CCMs into RL in a manner that enables the synthesis of contraction-aware, robust policies that achieve long-term optimality for high-dimensional nonlinear dynamical systems with minimal structural assumptions. Our approach is visualized in Figure 1. Our key idea is to use CCMs to define a reward function such that the optimal policy—trained using RL—exhibits both incremental asymptotic stability and long-term optimality. We implement this idea through our new algorithm, contraction-aware RL (CARL), which jointly learns a mapping from the system state to CCMs (referred to as the CMG) and a policy that maximizes rewards defined by CCMs sampled from the CMG. We use proximal policy optimization (PPO) [12] for RL optimization in this work, though in principle, our approach could be used with any RL algorithm. We empirically validate that CARL improves path-tracking performance compared to established baselines in four simulated environments of increasing dimensionality, ranging from a 4D Car to a 10D Quadrotor system. We also evaluate the robustness of CARL to approximation errors in the pretrained dynamics model (by considering simulated datasets of varying coverage) and to sim-to-real errors (by performing robot experiments with a TurtleBot3 Burger, a highly nonlinear Flapper Drone, and a high-dimensional complex Jethexa hexapod). As for our theoretical rationale, we prove the incremental asymptotic stability of an optimal policy trained by our algorithm and formally characterize the trade-off between long-term optimality and incremental exponential stability. We additionally provide conditions under which path-tracking policies, including those produced by CARL, are robust to approximation errors in the pretrained dynamics.

A. Contributions

Our contributions are summarized as follows:

- 1) We propose a contraction-aware RL algorithm (CARL) that uses CCMs and RL to synthesize a robust path-tracking policy that enhances long-term optimality.
- 2) We formally prove that the policy optimized by CARL exhibits incremental asymptotic stability and can also achieve incremental exponential stability by modulating the planning horizon governed by the discount factor, and establish conditions under which the optimized policy remains robust to approximation errors in the pretrained dynamics.

TABLE I: Summary of notation.

Symbol	Description
<i>Coordinates & Kinematics</i>	
p, v, a	Position, velocity, and acceleration
ϕ, θ, ψ	Euler angles: roll, pitch, and yaw
$\tilde{m}, g, \tilde{f}$	Mass, gravitational acceleration, and thrust
$\mathcal{I}$	Moment of inertia
<i>System Dynamics</i>	
$x \in \mathcal{X}, u \in \mathcal{U}$	State and control with their associated spaces
$x^* \in \mathcal{X}, u^* \in \mathcal{U}$	Reference state and control with their spaces
δx	Differential state (virtual displacement)
$f(x), B(x)$	Drift dynamics and actuation matrix
$\hat{f}_\xi(x), \hat{B}_\zeta(x)$	Approximate f and B with parameters ξ and ζ
<i>Contraction Theory</i>	
$M(x)$	Control contraction metrics
$\mathcal{V}(x, \delta x)$	Differential Lyapunov function
<i>Reinforcement Learning</i>	
$s \in \mathcal{S}$	RL state $s_t = (x_t, x_{t:t+N-1}^*, u_t^*)$
N	A fixed look-ahead horizon
$P(s' s, u)$	Transition probability density function
$R(s, u)$	Reward function
γ	Discount factor
$\pi_\vartheta(u s)$	Stochastic policy with parameters ϑ
G_t	Expected return, $G_t = \sum_{k=0}^{\infty} \gamma^k R(s_{t+k}, u_{t+k})$
$V_\varphi^{\pi_\vartheta}(s)$	State-value function for π_ϑ with parameters φ
$Q^{\pi_\vartheta}(s, u)$	Action-value function for π_ϑ
d_0	Initial state distribution, $s_0 \sim d_0$
$d^{\pi_\vartheta}(s, u)$	Discounted state-control occupancy for policy π_ϑ

- 3) We validate the advantages of prioritizing long-term optimality by CARL in both simulated environments and on diverse robotic platforms.

B. Organization

We review related work in Section II and provide preliminaries in Section III. We present CARL in Section IV and provide a theoretical analysis in Section V. We present empirical results from simulated experiments in Section VI and three real-world robot experiments in Section VII (wheeled), Section VIII (aerial), and Section IX (legged). We discuss concluding thoughts, limitations, and future work in Section X.

C. Notation

Since this work combines concepts from contraction theory and RL, we provide a comprehensive summary of the notation used throughout the paper in Table I.

II. RELATED WORKS

A. Non-learning Synthesis of Contracting Policies

Early works on CCMs focused on analytically identifying them in an offline phase and computing corresponding controls in an online phase. For example, [1] uses the dual metric of CCMs with SoS programming to solve an infinite-dimensional convex feasibility problem with the CCM conditions. Then,

they define the differential feedback gain and path-integrate it to obtain the control that drives the system dynamics to contract. [3] improves the method in [1] by additionally considering CCMs that minimize the area of the robust control invariant tube (i.e., they balance between contraction rate and overshoot) and using quadratic programming to find the most efficient control (i.e., control with lowest ℓ_2 -norm) for system contraction. However, these approaches require extensive computation to minimize geodesics, which can be computationally intractable for real-time deployment. Furthermore, the computational complexity of the SoS optimization scales exponentially with the problem dimension, restricting its use to small-scale, low-dimensional applications.

To address the intractability of computing exact geodesics while robustly accounting for external disturbances, [4] proposes the ConVex optimization-based Stochastic steady-state Tracking Error Minimization (CV-STEM) framework. This framework identifies contraction metrics that minimize an upper bound on the steady-state mean-squared tracking error of the system, and then uses them to derive a nonlinear counterpart of the linear quadratic regulator (LQR) feedback law. This controller design guarantees exponential boundedness of the tracking error and exhibits L_2 -robustness properties, while avoiding the need to compute minimizing geodesics. While this approach is more tractable than prior work, it remains computationally challenging for hardware-constrained robots. To further mitigate this real-time computational burden, [13] proposes a two-stage approach: they first compute the contraction metric and controls using CV-STEM in an offline phase, and subsequently employ supervised learning to distill these controls into a neural network policy for efficient real-time deployment. However, this approach requires a dense dataset for successful distillation and may not scale to high-dimensional systems, as finding contraction metrics is time-consuming and the volume of data required for successful distillation increases with state dimension.

While contraction theory offers a rigorous framework for nonlinear stability, classical linear control approaches remain standard for practical path-tracking applications. The most prominent example is LQR [14], which addresses the path-tracking problem by finding an optimal constant gain matrix with respect to the linearized error dynamics. Given that LQR relies on linearization of error dynamics and known system dynamics, Richards et al. [9] propose to approximate the system dynamics using neural networks and apply the state-dependent coefficient factorization (i.e., $f(x) = A(x)x$) to formulate the exact nonlinear error dynamics. These error dynamics are then used to solve the state-dependent Riccati equation in real-time to find stabilizing controllers. This approach has been shown to achieve a larger region of attraction (RoA) than standard LQR [9], [15]. However, [9] lacks formal guarantees for certifying the exact size of the RoA and is only locally stabilizing for nonlinear system dynamics. Additionally, LQR-based approaches require computationally intensive real-time optimization (as we show in Table III).

See [16] and the references therein for a comprehensive survey of these, and other, non-learning methods for synthesizing contracting policies.

B. Learning-based Synthesis of Contracting Policies

To mitigate the efficiency, scalability, and stability limitations of non-learning approaches, Sun et al. [8] propose a gradient-based learning approach for synthesizing policies with CCMs. More specifically, their approach learns neural networks that model a CCM generator and corresponding contracting policy. To train the neural networks, they first use a known dynamics model to evaluate the violation of CCM conditions for current outputs from the CMG and policy. Then, these violations are used as a loss function for simultaneously updating the CMG and policy. While this approach addresses efficiency, scalability, and stability limitations, it only focuses on satisfying the CCM conditions in a pointwise manner, which again does not ensure long-term optimality.

[10] recently proposed a modular control architecture that guarantees exponential stability within RL frameworks. They use a coordinate transformation to linearize nonlinear contraction constraints into algebraically solvable linear constraints, and apply these constraints to the weights of the policy's neural network. Specifically, they restrict the signs of the neural network's weights (e.g., to be strictly positive or negative) to satisfy algebraic constraints, ensuring the policy generates controls that drive the system to contract. However, the existence of the necessary coordinate transformation is not guaranteed, which may limit the method's applicability to general high-dimensional nonlinear systems. [11] employs PPO to synthesize a contracting policy by incorporating a contraction penalty as an auxiliary reward. They also analyze the worst-case contraction rate under the dynamics mismatch from a sim-to-real transfer. However, they do not provide a formal justification for how optimizing this augmented reward function leads to the actual synthesis of a contracting policy.

III. PRELIMINARIES

A. Dynamical System

We consider control-affine nonlinear systems given by

$$\dot{x}(t) = f(x(t)) + B(x(t))u(t), \quad (1)$$

where $x(t) \in \mathcal{X} \subset \mathbb{R}^n$ with $\mathcal{X}$ a compact state set, $u(t) \in \mathcal{U} \subset \mathbb{R}^m$ with $\mathcal{U}$ a compact control set, $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ denotes the smooth nonlinear drift dynamics, and $B : \mathbb{R}^n \rightarrow \mathbb{R}^{n \times m}$ is the smooth actuation matrix function. For notational simplicity, we omit the explicit time dependence to write $x(t)$ and $u(t)$ as x and u , unless the time argument is required for clarity.

B. Contraction Theory and Control Contraction Metrics

Contraction theory [2] provides a framework for analyzing the convergence between solution trajectories satisfying Equation (1) by leveraging the differential form of Lyapunov functions. Specifically, contraction theory reformulates Lyapunov stability conditions by employing a quadratic Lyapunov function defined over differential dynamics as follows:

$$\mathcal{V}(x(t), \delta x(t)) = \delta x(t)^\top M(x(t)) \delta x(t), \quad (2)$$

where $\delta x(t)$ is the differential state (i.e., virtual displacement) between two solution trajectories at time t , and $M(x(t)) \succ 0$ is

the symmetric positive definite matrix defining the contraction metric. When the time derivative of $\mathcal{V}$ satisfies $\dot{\mathcal{V}} \leq -2\lambda\mathcal{V}$ for some $\lambda > 0$, the differential dynamics of $\delta x(t)$ are guaranteed to decay exponentially at rate λ , which implies incremental exponential stability of the system. Specifically, let the contraction metric be bounded by $mI \preceq M \preceq \bar{m}I$, where $\bar{m}, m \in \mathbb{R}$ satisfy $0 < m \leq \bar{m}$ and I is the identity matrix. Then, the following exponential stability holds:

$$\|\delta x(t)\|_2 \leq \sqrt{\frac{\bar{m}}{m}} \|\delta x(t_0)\|_2 \cdot e^{-\lambda(t-t_0)}, \quad (3)$$

where $\sqrt{\bar{m}/m}$ accounts the overshoot.

[1] extends contraction theory to constructive closed-loop nonlinear control design. Formally, the system in Equation (1) with a control policy $u(t) = \pi_c(x(t), t) + v(t)$ (where $v(t)$ is an arbitrary function) is said to be contracting if there exists a matrix $M(x) \succ 0$ that satisfies

$$\dot{M} + 2\text{sym}(M(A + BK)) + 2\lambda M \preceq 0, \quad (4)$$

where we omit the input argument for clarity, $\text{sym}(\cdot)$ is the symmetric part of the input argument (i.e., $\text{sym}(A) = (A + A^\top)/2$), $A(x, u) = \frac{\partial f}{\partial x} + \sum_{i=1}^m u_i \frac{\partial b_i}{\partial x}$ is the Jacobian of the drift dynamics, $K(x, t) = \frac{\partial \pi_c}{\partial x}$ represents the differential feedback gain associated with the policy, and $\lambda > 0$ specifies the contraction rate. A sufficient condition for the existence of a contracting policy satisfying Equation (4) is given as follows:

$$B_\perp^\top (-\partial_f W(x) + 2\text{sym}(\frac{\partial W(x)}{\partial x} f(x)) + 2\lambda W(x)) B_\perp \prec 0, \quad (5)$$

$$B_\perp^\top (\partial_{b_j} W(x) - 2\text{sym}(\frac{\partial b_j(x)}{\partial x} W(x))) B_\perp = 0, \quad j = 1, \dots, m. \quad (6)$$

Here, $W(x) = M^{-1}(x)$ denotes the dual metric of M , $\partial_f W(x) = \sum_{i=1}^n f_i \frac{\partial W}{\partial x_i}$ represents the Lie derivative of the matrix-valued function $W(x)$ along the vector $f \in \mathbb{R}^n$, and $B_\perp$ denotes the matrix whose columns span the null space of the actuation matrix $B^\top$. The conditions above ensure that the dynamics evolving orthogonally to the span of actuated directions (i.e., directions that cannot be controlled by u) must naturally contract. A matrix M that satisfies Equations (4) to (6) defines a control contraction metric, and the satisfaction of these conditions ensures that the policy is guaranteed to exhibit incremental exponential convergence to the reference trajectory, serving as a strong certificate of stability.

C. Reinforcement Learning

Reinforcement learning (RL) addresses sequential decision-making under uncertainty through interactions with environments. In this work, we formulate the path-tracking problem as a Markov decision process (MDP) [17].

1) *Markov decision process*: An MDP is defined by a tuple $(\mathcal{S}, \mathcal{U}, P, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{U}$ is the control space, $P : \mathcal{S} \times \mathcal{U} \times \mathcal{S} \rightarrow [0, \infty)$ is the probability density of the next state being $s' \in \mathcal{S}$ given the current state $s \in \mathcal{S}$ and control input $u \in \mathcal{U}$, $R : \mathcal{S} \times \mathcal{U} \rightarrow \mathbb{R}$ is the reward function, and $\gamma \in [0, 1)$ is the discount factor. At time step t , the agent executes control $u_t \in \mathcal{U}$ given the current state $s_t \in \mathcal{S}$, after which the system transitions to state $s_{t+1} \in \mathcal{S}$ and the agent

receives reward $r_t = R(s_t, u_t)$. The expected return at time t is defined as $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$.

For our path-tracking problem, we define the state space as $\mathcal{S} = \mathcal{X} \times \mathcal{X}^N \times \mathcal{U}$, where $\mathcal{X}$ and $\mathcal{U}$ are defined as in Section III-A, and N is a fixed look-ahead horizon. The state at time t is defined by the agent state, the reference state, and the reference control such that $s_t = (x_t, x_{t:t+N-1}^*, u_t^*)$. Our goal is to optimize a policy $\pi : \mathcal{S} \times \mathcal{U} \rightarrow [0, \infty)$, which defines a probability density over continuous controls $\mathcal{U}$ and maximizes the expected return.

2) *Policy Optimization*: Policy gradient methods are commonly used for policy optimization when considering high-dimensional and continuous state and control spaces. Let π_ϑ be a policy parameterized by ϑ and $V^{\pi_\vartheta}(s) = \mathbb{E}_{\pi_\vartheta}[G_t | s_t = s]$ be its corresponding state-value function. The goal is to maximize the following performance objective, where $d_0 : \mathcal{S} \rightarrow [0, \infty)$ is the initial state distribution:

$$J(\vartheta) = \mathbb{E}_{s_0 \sim d_0} [V^{\pi_\vartheta}(s_0)]. \quad (7)$$

Proximal policy optimization (PPO) [12] is a widely used policy optimization algorithm based on an actor-critic architecture that has shown strong performance and efficiency across a wide range of robotic tasks [18]. In this work, we employ PPO for policy optimization, which often includes entropy, denoted by $\mathcal{H}(\pi_\vartheta(\cdot | s))$, in the performance objective as follows:

$$J(\vartheta) \leftarrow J(\vartheta) + \beta_\pi \mathbb{E}_{s \sim d^{\pi_\vartheta}} [\mathcal{H}(\pi_\vartheta(\cdot | s))], \quad (8)$$

where $\beta_\pi > 0$ is a hyperparameter and d^π is the discounted state occupancy induced by π . The entropy regularization is known to smooth the optimization landscape [19] by encouraging exploration of a policy.

IV. MAIN RESULT: AN ALGORITHM FOR CONTRACTION-AWARE REINFORCEMENT LEARNING

We propose a new algorithm, contraction-aware RL (CARL), for synthesizing a robust path-tracking policy that achieves incremental asymptotic stability and long-term optimality. CARL is visualized in Figure 2 and outlined in Algorithm 1. It is composed of two primary steps: pretraining a dynamics model and then jointly learning a CCM generator (CMG) and policy.

Our empirical results (see Sections VII to IX) demonstrate that the resulting policy can then be deployed on real-world robots in a zero-shot manner. We describe each step of CARL in more detail below.

A. Pretraining a Dynamics Model

We first pretrain an approximate dynamics model, which we later use to train a CMG (see Section IV-B). We approximate the dynamics by learning approximations of the drift dynamics f , the actuation matrix function B , and the annihilator matrix of the actuation matrix $B_\perp$. Following Richards et al. [9], we employ neural networks to model f and B and use singular value decomposition (SVD) to compute $B_\perp$. Specifically, given a dataset $\mathcal{D} = \{(\hat{x}_i, x_i, u_i)\}_{i=1}^{N_{\text{data}}}$, we simultaneously

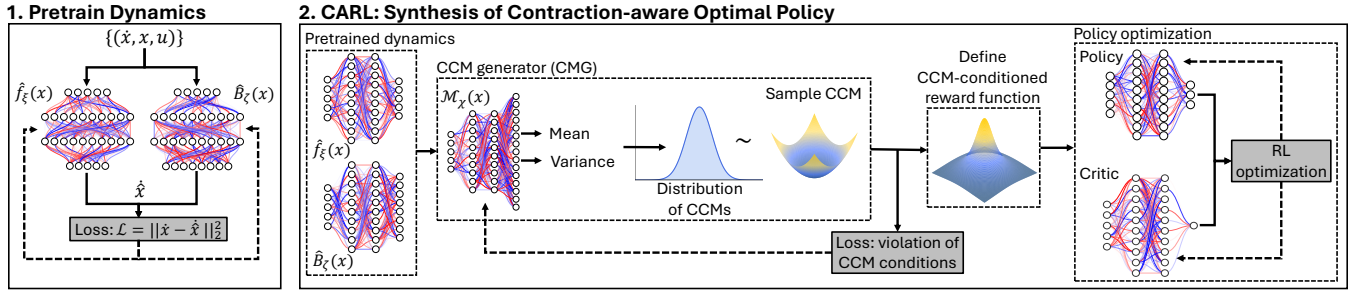


Fig. 2: We present a comprehensive overview of our proposed algorithm. We first pretrain a dynamics model using an offline dataset that contains tuples of $(\dot{x}, x, u)$. Then, we jointly train a CCM generator (CMG), using the pretrained dynamics model, and an RL policy, using a CCM-conditioned reward function that is evaluated by sampling the CMG. This formulation allows the RL policy to minimize cumulative tracking error with a stability guarantee.

Algorithm 1 Contraction-Aware RL (CARL)

Require: Offline data $\mathcal{D} = \{(\dot{x}_i, x_i, u_i)\}_{i=1}^{N_{\text{data}}}$, state $\mathcal{X}$, control space $\mathcal{U}$, N_{dynamics} , N_{CMG} , N_{policy}
Initialize: Neural network models $\hat{f}_\xi, \hat{B}_\zeta, \mathcal{M}_\chi, \pi_\vartheta$, and $V_\varphi^{\pi_\vartheta}$

```

1: /* Pretrain dynamics model */
2: for iteration = 1, 2, ...,  $N_{\text{dynamics}}$  do
3:   Sample batch  $d \sim \text{Unif}(\mathcal{D})$ 
4:   Update dynamics parameters  $\xi, \zeta$  with Eqn. 9 using  $d$ 
5: end for
6: /* Policy optimization */
7: for iteration = 1, 2, ...,  $N_{\text{CMG}}$  do
8:   Sample batch  $d \sim \text{Unif}(\mathcal{X}, \mathcal{U})$ 
9:   Update CMG parameter  $\chi$  with Eqn. 10 using  $d$ 
10:  for iteration = 1, 2, ...,  $N_{\text{policy}}$  do
11:    Collect rollouts using current policy  $\pi_\vartheta$ .
12:    Calculate CCM-conditioned rewards using CCMs  $M = \mathcal{M}_\chi(x)$ 
13:    Update policy parameters  $\vartheta$  and critic parameters  $\varphi$ 
14:  end for
15: end for

```

learn two feed-forward networks, $\hat{f}_\xi$ and $\hat{B}_\zeta$, parameterized by ξ and ζ respectively, as follows:

$$\xi^*, \zeta^* = \underset{\xi, \zeta}{\operatorname{argmin}} \mathbb{E}_{(\dot{x}, x, u) \sim \mathcal{D}} \left[\left\| \dot{x} - \left(\hat{f}_\xi(x) + \hat{B}_\zeta(x)u \right) \right\|_2^2 \right]. \quad (9)$$

Then, the annihilator matrix of $\hat{B}^\top$ (i.e., $\hat{B}_\perp$) is computed using SVD, as it is spanned by the last $n - \text{rank}(\hat{B})$ columns of $\mathbb{U}$ in $\text{SVD}(\hat{B}) = \mathbb{U}\Sigma\mathbb{V}^\top$. For instance, for $\hat{B} \in \mathbb{R}^{3 \times 2}$ with $\text{rank}(\hat{B}) = 2$, the third column of $\mathbb{U} \in \mathbb{R}^{3 \times 3}$ spans the null space of $\hat{B}^\top$, which we use as the approximation of the annihilator matrix $\hat{B}_\perp$. We employ SVD to leverage existing code libraries, though a naive Gram–Schmidt process is a valid alternative for constructing the annihilator matrix.

B. Jointly Learning a CMG and a Policy

Given a pretrained dynamics model, we then simultaneously learn a CMG and a policy that maximizes rewards defined by the CMG. This process can be numerically unstable because the CMG is conditioned on the current policy. We address this issue by implementing a freeze-and-learn strategy, where for each update of the CMG, we freeze its parameters and

then perform several policy updates to allow the RL policy to adapt to the newly defined reward landscape. Note that the pretrained dynamics model is only used to train the CMG; it is not directly used for policy optimization.

1) *Learning a control contraction metric generator:* Given a pretrained dynamics model, we train a CMG $\mathcal{M}_\chi$, parameterized by χ , to generate CCMs for the current policy. Recall that CCMs are $n \times n$ matrices satisfying Equations (4) to (6), where n is the dimension of the state x , as defined in Section III-A. We assume a probabilistic model for the CMG, where CCMs are modeled as a random matrix with each element being an independent univariate Gaussian distribution. The CMG is then trained to output the mean and variance for each of those distributions—that is, the CMG outputs n^2 mean values and n^2 variances. We then sample a CCM for the current policy and state x as $M \sim \mathcal{M}_\chi(\cdot | x)$. Inspired by [8], [19], we use this probabilistic model for the CMG to train it with entropy regularization, which we use to encourage exploration in finding feasible CCMs and prevent premature convergence.

We train the CMG using the loss function from Sun et al. [8] with an additional value-conditioned entropy regularizer. Specifically, let $C_M(\cdot)$, $C_{W_1}(\cdot)$, and $C_{W_2}^j(\cdot)$ denote the left-hand side of Equations (4) and (5) and the j -th equality constraint in Equation (6) for $j = 1, \dots, m$, respectively. Then, the CMG is trained to minimize the violation of CCM conditions over the state and control spaces as follows:

$$\begin{aligned} \chi^* = \underset{\chi}{\operatorname{argmin}} \mathbb{E}_{\substack{x \sim \text{Unif}(\mathcal{X}), \\ u \sim \pi_\vartheta(\cdot | x), \\ M \sim \mathcal{M}_\chi(\cdot | x), \\ (x^*, u^*) \sim \mathcal{T}}} & \left[\mathcal{L}(M - \overline{m}I) + \mathcal{L}(C_M(M, x, u)) \right. \\ & + \mathcal{L}(C_{W_1}(M^{-1}, x)) + \sum_{j=1}^m \|C_{W_2}^j(M^{-1}, x)\|_F \\ & \left. - \alpha(V^{\pi_\vartheta}(s)) \cdot \mathcal{H}(\mathcal{M}_\chi(\cdot | x)) \right]. \end{aligned} \quad (10)$$

Here, $\mathcal{T}(x^*, u^*)$ is a distribution over a set of reference trajectories and $\mathcal{L}(A) = (z^\top A z) / (z^\top z)$ if $A \succ 0$ else 0 is a loss function that penalizes positiveness of the input matrix, where $z \in \mathbb{R}^n$ and $z_i \sim \text{Unif}(-1, 1)$. Additionally, $\|\cdot\|_F$ is the Frobenius matrix norm and $\mathcal{H}(\mathcal{M}_\chi(\cdot | x))$ is the entropy of the probability distribution $\mathcal{M}_\chi$. The strength of the entropy regularizer is modulated by the value function $V^{\pi_\vartheta}(s)$, where $\alpha(V^{\pi_\vartheta}(s)) = \beta_{\mathcal{M}} e^{-V^{\pi_\vartheta}(s)}$ with hyperparameter $\beta_{\mathcal{M}} > 0$. Intuitively, the value-conditioned entropy regularizer reduces

the degree of exploration by the CMG when the agent achieves high values and increases it when the agent achieves low values.

The auxiliary loss terms $\mathcal{L}(C_{W_1}(\cdot))$ and $\|C_{W_2}^j(\cdot)\|_F$ in Equation (10) are critical for convergence of the CMG, as directly optimizing $\mathcal{L}(C_M(\cdot))$ has been shown to struggle with convergence due to the bilinear coupling between CCMs and the policy [8]. Specifically, the CMG requires a stabilizing policy to determine accurate update directions, while the policy conversely relies on a high-fidelity CMG to guide its own optimization. The auxiliary loss terms resolve this bilinear coupling by guiding the CMG toward a structurally valid metric space that is independent of the policy. However, strict minimization of these auxiliary loss terms is not required because they represent sufficient, but not necessary, conditions for the contraction of the dynamics. Consequently, these terms function as regularizers for stable training.

In Equation (10), we additionally reduce the overshoot that is proportional to $\sqrt{\bar{m}/\underline{m}}$ (see Equation (3)). Specifically, we ensure the eigenvalues of M are within the range of $\underline{m}$ and $\bar{m}$ by adding $\underline{m}$ to the generated CCMs (i.e., $M = \mathcal{M}_\chi(x) + \underline{m}I$) and penalizing the violation of the upper bound through $\mathcal{L}(M - \bar{m}I)$.

2) Learning a control contraction metric-guided policy:

Prior works that synthesize contracting policies (see Section II) do not consider long-term optimality. Specifically, suppose there exist CCMs that satisfy the conditions in Equations (4) to (6). This guarantees that there exists a set of control inputs $\tilde{\mathcal{U}} \subseteq \mathcal{U}$ that ensures contraction [1]. While multiple choices of control $u \in \tilde{\mathcal{U}}$ may satisfy the CCM conditions under a given metric, they are not guaranteed to minimize the cumulative tracking error across the entire system trajectory. The key idea of CARL is to instead use the CMG to define a CCM-conditioned reward function, which we then optimize using RL to synthesize path-tracking policies that ensure incremental asymptotic stability and long-term optimality.

We specifically use the following CCM-conditioned reward function:

$$R(s) := \frac{1}{1 + \delta x^\top M \delta x} \in [0, 1], \quad (11)$$

where we approximate the infinitesimal displacement, δx , as the Euclidean displacement between x and x^* (i.e., $\delta x = x - x^*$) as in [8], since computing exact geodesics is computationally prohibitive. This approximation is valid because the reward approaches its maximum value of 1 upon contraction (i.e., $\delta x = 0$) and approaches 0 as $\delta x \rightarrow \infty$, effectively encouraging contraction. Additionally, we include the +1 term in the denominator to ensure the reward is bounded within $[0, 1]$, motivated by prior work that has shown bounded rewards can improve numerical stability of RL algorithms [20]–[22]. We deterministically generate M by defining it as the mean from the CMG $\mathcal{M}_\chi(x)$, avoiding the complexity of training with a stochastic reward function since a single valid CCM for each state is sufficient for contraction.

Our proposed reward function achieves two key properties. First, the reward uses CCMs to ensure the resulting optimization landscape is contraction-aware—that is, the reward function in Equation (11) encourages convergence to policies

that not only minimize cumulative tracking, but also do so in a way that contracts. Second, the reward is defined such that its maximization is equivalent to minimizing the cumulative tracking error, thus ensuring long-term optimality. We prove this equivalence in Section V-A.

While the reward defined in Equation (11) is intuitively and theoretically justified (see Section V for theory), in practice, we include an entropy regularization term, again motivated by prior work showing the benefit of entropy regularization for exploration in RL [19]. We also deterministically execute the trained stochastic policies for evaluation using the mean of the control distribution to better align our theoretical analysis (Section V), which assumes deterministic policies. [23] also empirically shows that deterministic policy evaluation generally outperforms stochastic evaluation.

V. THEORETICAL RESULTS:

BRIDGING RL AND CONTRACTION THEORY

This section provides theoretical justification for CARL. In Section V-A, we first prove the incremental asymptotic stability of an optimal policy for our CCM-conditioned reward, and demonstrate how the discount factor modulates the trade-off between long-term optimality and incremental exponential stability. In Section V-B, we then establish conditions under which that policy remains robust to approximation errors in the pretrained dynamics.

We consider an optimal policy π^* that maximizes the expected return for the reward defined in Equation (11) (i.e., without entropy regularization) to study the core idea of a CCM-conditioned reward. We also restrict our analysis to deterministic optimal policies to align with the deterministic formulation of CCMs [1]. This restriction is justified without loss of generality, since a deterministic stationary optimal policy is guaranteed to exist for infinite-horizon discounted MDPs with compact state-control spaces and smooth dynamics [17], [24]. Additionally, [25] theoretically shows that optimizing stochastic policies with proper annealing of stochasticity can converge to an optimal deterministic policy. All proofs are deferred to Appendix A.

We assume the following throughout our analysis.

Assumption 1: There exists at least one deterministic contracting policy π_c that satisfies Equation (4) with a rate of $\lambda > 0$. Hence, this policy π_c satisfies

$$\|\delta x(t_k)\|_M^2 \leq \|\delta x(t_0)\|_M^2 e^{-2\lambda k \Delta t}, \quad \forall k \in \mathbb{N}, \quad (12)$$

where Δt is a discrete time interval, t_0 is the initial time, $t_k = t_0 + k\Delta t$ is the current time, and $\|\delta x(t)\|_M^2 = \int_0^1 \frac{\partial \tilde{x}}{\partial \mu}^\top M(\tilde{x}(\mu, t)) \frac{\partial \tilde{x}}{\partial \mu} d\mu$ denotes the integration over the geodesics connecting $x(t)$ and $x^*(t)$, with $\tilde{x}(0, t) = x(t)$ and $\tilde{x}(1, t) = x^*(t)$.

A. Stability Analysis of a CCM-Conditioned Optimal Policy

Let us define a cost function as the path-tracking error under CCMs and its corresponding cumulative path-tracking error under a contracting policy π_c as follows:

$$C(s_t) := \|\delta x(t_0)\|_M^2, \quad V_C^{\pi_c}(s_t) := \sum_{k=0}^{\infty} \gamma^k C(s_{t+k}), \quad (13)$$

and define the value function and objective for the reward function R in Equation (11) for a policy π as follows:

$$V_R^\pi(s) := \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R(s_{t+k}) \mid s_t = s \right], \quad (14)$$

$$J_R(\pi) := \mathbb{E}_{s_0 \sim d_0} [V_R^\pi(s_0)]. \quad (15)$$

Let π^* be a deterministic optimal policy for J_R as follows:

$$\pi^* := \operatorname{argmax}_{\pi} J_R(\pi). \quad (16)$$

To facilitate an analysis within a minimization framework, we introduce another cost function $\tilde{C}$ in terms of R and its corresponding value function and objective as follows:

$$\tilde{C}(s) := 1 - R(s) \in [0, 1], \quad (17)$$

$$V_{\tilde{C}}^\pi(s) := \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k \tilde{C}(s_{t+k}) \mid s_t = s \right], \quad (18)$$

$$J_{\tilde{C}}(\pi) := \mathbb{E}_{s_0 \sim d_0} [V_{\tilde{C}}^\pi(s_0)]. \quad (19)$$

Then, we show the equivalence of minimizing the cost objective $J_{\tilde{C}}(\pi)$ and maximizing the reward objective $J_R(\pi)$.

Lemma 1: (Equivalence of Cost Minimization and Reward Maximization). Let the reward and cost objectives be defined as in Equation (15) and Equation (19), respectively. Then, for a discount factor $\gamma \in [0, 1)$, the following equivalence holds

$$\pi^* = \operatorname{argmin}_{\pi} J_{\tilde{C}}(\pi) = \operatorname{argmax}_{\pi} J_R(\pi), \quad (20)$$

assuming consistent tie-breaking rules.

We also establish the contraction of π_c with respect to $\tilde{C}$.

Lemma 2: (Contraction with respect to $\tilde{C}$). Suppose there exists at least one policy π_c that contracts with respect to C with rate $\lambda > 0$ as in Assumption 1. Then, there exists a rate $\tilde{\lambda} \in (0, \lambda)$ such that π_c contracts with respect to $\tilde{C}$ as follows:

$$\tilde{C}(s_{t+k}) \leq \tilde{C}(s_t) e^{-2\tilde{\lambda}k\Delta t}, \quad \forall k \in \mathbb{N}. \quad (21)$$

The reward value function V_R^π , reward objective $J_R(\pi)$, and Lemma 1 require a discount factor $\gamma < 1$ for finiteness under an infinite horizon, which poses a challenge for stability analysis of typical RL algorithms. However, due to our integration of CCMs with RL, we are able to establish that the optimal policy π^* satisfying Equation (16) guarantees incremental asymptotic stability for any discount factor $\gamma \in (0, 1)$.

Theorem 1: (Incremental Asymptotic Stability). Suppose there exists at least one policy π_c that contracts with respect to C with rate $\lambda > 0$ as in Assumption 1. Then, for any discount factor $\gamma \in (0, 1)$, a deterministic optimal policy π^* satisfying Equation (16) renders the closed-loop system incrementally asymptotically stable:

$$\lim_{k \rightarrow \infty} \|\delta x(t_k)\|_M^2 = 0, \quad \forall \delta x(t_0) \in \mathbb{R}^n. \quad (22)$$

Finally, we establish incremental exponential stability as follows.

Theorem 2: (Incremental Exponential Stability). Suppose there exists at least one policy π_c that contracts with respect to C with rate $\lambda > 0$ as in Assumption 1. Then, for a discount factor $\gamma < \min(\bar{\gamma}, 1)$, there exists a contraction factor $\bar{\rho} \in (0, 1)$ such that a deterministic optimal policy π^* satisfying

Equation (16) renders the closed-loop system incrementally exponentially stable:

$$\|\delta x(t_k)\|_M^2 \leq \|\delta x(t_0)\|_M^2 \bar{\rho}^k, \quad \forall k \in \mathbb{N}, \delta x(t_0) \in \mathbb{R}^n, \quad (23)$$

where $\bar{\gamma} := e^{2\tilde{\lambda}\Delta t} - 1 > 0$ and $\tilde{\lambda}$ is the contraction rate induced by π_c with respect to the cost function $\tilde{C}$ as in Lemma 2.

Theorem 2 demonstrates that CARL can directly trade-off between long-term optimality and the strength of its stability guarantee through the modulation of the discount factor γ . This is intuitive, since minimizing future errors (i.e., a long-term optimal strategy) may sacrifice near-term performance, causing overshoots that violate exponential stability. If the threshold $\bar{\gamma} \geq 1$, Theorem 2 guarantees that the optimal policy π^* is both long-term optimal and exponentially stabilizing for discount factor $\gamma \rightarrow 1$. Note that Theorem 1 and Theorem 2 are direct results of using our CCM-integrated reward function.

B. Robustness of RL-trained Optimal Policy

Finally, we analyze the robustness of π^* under the approximate dynamics model learned from Equation (9). To this end, let us define the dynamics modeling error as

$$\mathbf{d}(x, u) := f(x) + B(x)u - \hat{f}_\xi(x) - \hat{B}_\zeta(x)u. \quad (24)$$

Also, for any trajectory segment of length κ , we define the truncated ℓ^2 norm weighted by the contraction metric as

$$\|(\delta x)_\kappa\|_{\ell_M^2} := \sqrt{\sum_{k=0}^{\kappa-1} \|\delta x(t_k)\|_M^2}. \quad (25)$$

The following theorem provides an extension of the result presented in [26].

Theorem 3: (Robustness of RL-trained Policies). Suppose that both the contracting policy π_c of Assumption 1 and the deterministic optimal policy π^* of Equation (16) are optimized using CCMs based on a pretrained dynamics model $\hat{x} = \hat{f}_\xi(x) + \hat{B}_\zeta(x)u$. Given calibration trajectories $\{x_j(t), u_j(t)\}_{j=1}^{N_{\text{cal}}}$ not used for pretraining $\hat{x}$, we define the nonconformity score quantifying the model mismatch as follows:

$$\bar{\mathbf{d}}_j := \sup_{t \geq t_0} \|\mathbf{d}(x_j(t), u_j(t))\|, \quad j = 1, \dots, N_{\text{cal}}. \quad (26)$$

If these scores and the score computed with the contracting policy π_c are exchangeable and π^* achieves a lower cumulative tracking error than π_c , then a test trajectory executed using π^* satisfies the following bound on the time-averaged ℓ^2 -norm:

$$\mathbb{P} \left[\lim_{\kappa \rightarrow \infty} \frac{\|(\delta x)_\kappa\|_{\ell_M^2}}{\kappa} \leq \frac{\sqrt{m} \mathbf{Q}_{1-\varepsilon}}{\lambda} \right] \geq 1 - \varepsilon. \quad (27)$$

Here, $\varepsilon \in (0, 1)$ is a user-specified risk tolerance level, $\mathbf{Q}_{1-\varepsilon}$ is the $(1 - \varepsilon)$ -th quantile of the empirical distribution of $\{\bar{\mathbf{d}}_j\}_{j=1}^{N_{\text{cal}}} \cup \infty$, and x is the state of Equation (1) under the policy π^* . Note that the probability measure $\mathbb{P}$ is taken over the randomness of the calibration trajectories and the test trajectory [27].

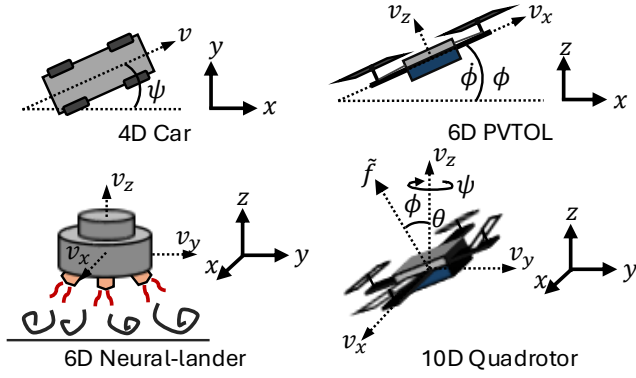


Fig. 3: Notional illustrations of the robots used in our simulated experiments and their underlying dynamics.

VI. EXPERIMENTAL RESULTS: SIMULATION

We conducted experiments in simulated environments to evaluate CARL's performance and robustness, implied in Theorems 1 and 3, relative to a set of relevant baselines.

A. Simulated Environments

We considered four widely used robot models for path-tracking experiments [8], [28]–[30]: 4D Car, 6D PVTOL, 6D Neural-lander, and 10D Quadrotor. We visualize these models in Figure 3 and define the corresponding dynamics below. These dynamics were only used to define our simulation environments—we assume they are unknown for all considered algorithms.

1) *4D Car*: The state of the car is defined as $x := (p_x, p_y, v, \psi)$, where p_x and p_y define the 2D position, v is the linear speed along the yaw angle, and ψ is the yaw angle. The control input is defined as $u := (a, \omega)$, where a is the linear acceleration and ω is the angular velocity. The dynamics are expressed as follows:

$$f(x) = \begin{bmatrix} v \cos(\psi) \\ v \sin(\psi) \\ 0 \\ 0 \end{bmatrix}, \quad B(x) = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}. \quad (28)$$

This design is a relaxation of the classic Dubins car, as it allows the linear velocity to vary (rather than being fixed). This is the environment with the lowest dimensionality and dynamic complexity that we consider.

2) *6D PVTOL*: We adopted the PVTOL model from [8], [29], [30]. The state of the PVTOL is defined as $x := (p_x, p_z, \phi, v_x, v_z, \dot{\phi})$, where p_x and p_z define the 2D position, ϕ is the roll angle, v_x and v_z are the linear velocities, and $\dot{\phi}$ is the roll angle rate. The control input is defined as $u := (\tilde{f}_1, \tilde{f}_2)$, where $\tilde{f}_1$ and $\tilde{f}_2$ are the thrust of two rotors that controls the v_z and ϕ . The dynamics are expressed as follows:

$$f(x) = \begin{bmatrix} v_x \cos(\phi) - v_z \sin(\phi) \\ v_x \sin(\phi) + v_z \cos(\phi) \\ \dot{\phi} \\ v_z \dot{\phi} - g \sin(\phi) \\ -v_x \dot{\phi} - \cos(\phi) \\ 0 \end{bmatrix}, \quad B(x) = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 1/\tilde{m} & 1/\tilde{m} \\ l/\mathcal{I} & -l/\mathcal{I} \end{bmatrix}, \quad (29)$$

where g is the gravitational acceleration, $\tilde{m}$ is the mass, l is the rotor arm length, and $\mathcal{I}$ is the moment of inertia.

3) *6D Neural-lander*: We adopted the Neural-lander model from [8], [28]. The state of the Neural-lander is defined as $x := (p_x, p_y, p_z, v_x, v_y, v_z)$, where p_x , p_y , and p_z define the 3D position, and v_x , v_y , and v_z are the 3D linear velocities. The control input is defined as the 3D linear accelerations as $u := (a_x, a_y, a_z)$. The dynamics are expressed as follows:

$$f(x) = \begin{bmatrix} v_x \\ v_y \\ v_z \\ F_1/\tilde{m} \\ F_2/\tilde{m} \\ F_3/\tilde{m} - g \end{bmatrix}, \quad B(x) = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad (30)$$

where $F_i := F_i(p_z, v_x, v_y, v_z)$ are neural network functions that were trained to predict the ground effect for each dimension. [28] developed this environment to consider the ground effect of a flying robot when landing.

4) *10D Quadrotor*: We adopted the Quadrotor model from [8], [30]. The state of the Quadrotor is defined as $x := (p_x, p_y, p_z, v_x, v_y, v_z, \tilde{f}, \phi, \theta, \psi)$, where p_x , p_y , and p_z define the 3D position, v_x , v_y , and v_z define the 3D linear velocities, $\tilde{f}$ is the thrust, and ϕ , θ , and ψ are the attitude. The control input is defined as the rate of change of thrust and attitude as $u := (\dot{\tilde{f}}, \dot{\phi}, \dot{\theta}, \dot{\psi})$. The dynamics are expressed as follows:

$$f(x) = \begin{bmatrix} v_x \\ v_y \\ v_z \\ \tilde{f} \sin(\psi) \\ -\tilde{f} \cos(\psi) \sin(\phi) \\ \tilde{f} \cos(\psi) \cos(\phi) - g \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad B(x) = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (31)$$

B. Baseline Algorithms

We considered the following baseline algorithms for comparison:

- 1) Certified control using contraction metric (C3M) [8],
- 2) Proximal policy optimization (PPO) [12],
- 3) State-dependent LQR (SD-LQR) [9],
- 4) Standard LQR [14].

C3M is a learning-based algorithm that simultaneously identifies CCMs and a corresponding contracting policy using two neural networks. PPO is a widely used actor-critic algorithm that CARL is based on. SD-LQR builds on LQR by introducing exact nonlinear error dynamics using state-dependent coefficients; it solves the state-dependent Riccati equation, which practically results in a larger region of attraction than LQR [9], [15].

The RL algorithms (CARL and PPO) take $s_t = (x_t, x_{t:t+N-1}^*, u_t^*)$ as input to generate controls, while the control-theoretic baselines (C3M, SD-LQR, and LQR) take

TABLE II: List of key hyperparameters used in the experiments. The CCM parameters λ and $\bar{m}, \underline{m}$ govern the speed and transient overshoot of the contraction, respectively, while the RL parameters are standard configurations for general policy optimization.

Parameter	Symbol	Value
CCM parameters		
Target contraction rate	λ	0.5
CCM upper bound	$\bar{m}$	10.0
CCM lower bound	$\underline{m}$	0.1
Entropy-regularization	$\beta_{\mathcal{M}}$	0.01
RL parameters		
Discount factor	γ	0.99
Generalized advantage estimator	-	0.95
Target KL divergence	-	0.01
Entropy-regularization	β_{π}	0.001

$s_t = (x_t, x_t^*, u_t^*)$ since their control laws inherently embed the system dynamics for path-tracking. The baseline PPO is optimized using the reward function in Equation (11) with entropy regularization and CCMs replaced by the identity matrix (i.e., $M = I$). We list the key hyperparameters used for CARL and the baselines in Table II.

We adopted λ , $\bar{m}$, and $\underline{m}$ from Sun et al. [8] which considers similar dynamical systems, and tuned $\beta_{\mathcal{M}}$, β_{π} , and the target KL divergence using a grid search.

C. Experiment Details

1) *Reference trajectory generation*: We generated reference trajectories following Sun et al. [8]. Specifically, we defined a compact set of initial reference states $\mathcal{X}_0^* \subset \mathcal{X}$, from which the initial state of the reference trajectory is uniformly sampled, i.e., $x^*(0) \sim \text{Unif}(\mathcal{X}_0^*)$. A reference control $u^*(t)$ was then constructed as a convex combination of N_{ref} sinusoidal functions as follows:

$$u^*(t) = \sum_{i=1}^{N_{\text{ref}}} w_i \sin\left(2\pi \varrho_i \frac{t}{T_{\text{max}}}\right), \quad (32)$$

where ϱ_i are fixed frequencies, w_i are normalized weights that sum to 1, and T_{max} is the maximum time horizon. We set $N_{\text{ref}} = 10$, defined the frequencies as $\varrho_i = i$, and sampled weights $w_i \sim \text{Unif}(0, 1)$, which were subsequently normalized. We ensured the trajectory remains within the predefined compact set $\mathcal{X}$ by truncating the episode when the Cartesian coordinates exceed the bounds and clipping the remaining components. This process may result in reference trajectories of varying time lengths.

2) *Pretrained dynamics model*: We constructed three datasets of differing coverage of the state and control spaces. Our baseline data contained $N_{\text{data}} = 100,000$ homogeneous samples of $(\dot{x}, x, u)$ data points, generated by uniformly sampling $N_{\text{data}}/N_{\text{ctr}}$ states from $\mathcal{X}$, pairing each with $N_{\text{ctr}} = 1$ control inputs uniformly sampled from $\mathcal{U}$, and computing corresponding dynamics $\dot{x}$ using the corresponding nominal dynamics model in Section VI-A. This procedure follows Richards et al. [9]. We also constructed control-focused data generated using the same procedure but with $N_{\text{ctr}} = 3$ to provide better coverage of the control space, but with less

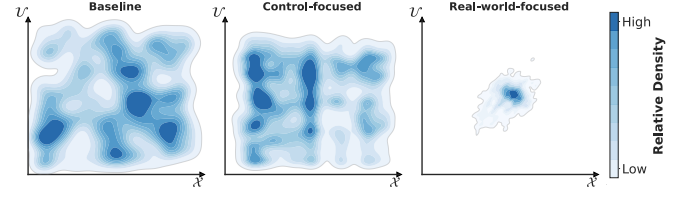


Fig. 4: Notional density plots of the sampled state ($\mathcal{X}$) and control ($\mathcal{U}$) spaces for pretraining dynamics models. The baseline dataset shows a broad, uniform distribution in the state space with limited coverage in the control space. The real-world dataset is constrained to a highly specific region. The control-focused dataset spans the control space extensively but remains limited in the state space.

coverage of the state space. Finally, we constructed real-world-focused data that better reflects real-world constraints, containing only 10,000 samples of reference trajectories generated by adding Gaussian noise to the reference controls. We provide notional illustrations of these datasets in Figure 4.

We used these datasets to pretrain models of the system dynamics, $\hat{f}_{\xi}$ and $\hat{B}_{\zeta}$ (see Section IV-A). Each model was represented by a feed-forward neural network with hidden layers of size (256, 256) and Tanh activation functions. We refer to the dynamics models trained with baseline, control-focused, and real-world-focused data as the baseline, control-focused, and real-world-focused dynamics models, respectively.

3) *CMG network*: Following Sun et al. [8], we modeled the CMG using a feed-forward neural network with hidden layers of size (256, 256) and Tanh activation functions. However, unlike [8], we introduced dual output layers to generate the mean and variance for parameterizing univariate Gaussian distributions over CCMs.

4) *Policy network*: Following Sun et al. [8], we modeled the policy for CARL, C3M, and PPO using two feed-forward neural networks with hidden layers of size (128) and Tanh activation functions, parameterized by $\theta = \{\theta_1, \theta_2\}$, and defined controls as follows:

$$u_t = u_t^* + w_2^{\top} \cdot \tanh(w_1 \cdot (x_t - x_t^*)), \quad (33)$$

where u^* is the reference control used to generate the next reference state, $w_1 = \theta_1(x_t, x_{t:t+N-1}^*) \in \mathbb{R}^{n \times n}$, and $w_2 = \theta_2(x_t, x_{t:t+N-1}^*) \in \mathbb{R}^{n \times m}$. C3M synthesizes controls using Equation (33) with w_1 and w_2 taking (x_t, x_t^*) .

5) *Evaluation metric*: We first define the normalized tracking error for time $t_k = k\Delta t$, with discrete time interval Δt , as

$$\|e(t_k)\|_2 := \frac{\|x(k\Delta t) - x^*(k\Delta t)\|_2}{\|x(0) - x^*(0)\|_2}. \quad (34)$$

Then, similar to [8], [9], we introduce a metric for path-tracking performance under varying lengths of reference trajectories via the modified area under the curve (mAUC) of the normalized tracking error as follows:

$$\text{mAUC} := \frac{L_{\text{max}}}{L} \sum_{k=0}^L \|e(t_k)\|_2, \quad (35)$$

where $L_{\text{max}} = T_{\text{max}}/\Delta t$, $L = T/\Delta t$, and T is the time taken for the executed trajectory. We use mAUC as a primary path-tracking performance indicator, as it captures the cumulative tracking error for varying lengths of reference trajectories.

TABLE III: **Evaluation results with baseline dynamics models.** Mean and 95% confidence intervals of mAUC over 5 seeds are reported with mean per-step inference time in milliseconds (ms). The best result for each environment is bolded.

Method	mAUC (lower is better)				Per-step Inference Time (ms)			
	Car	PVTOL	Neural-lander	Quadrotor	Car	PVTOL	Neural-lander	Quadrotor
CARL (Ours)	1.07 ± 0.05	7.49 ± 0.84	2.47 ± 0.42	5.55 ± 1.36	0.1	0.1	0.1	0.1
C3M [8]	1.06 ± 0.07	29.6 ± 6.4	2.63 ± 0.44	6.97 ± 1.72	0.1	0.1	0.1	0.1
PPO [12]	1.56 ± 0.18	12.7 ± 3.8	2.64 ± 0.48	5.84 ± 1.13	0.1	0.1	0.1	0.1
SD-LQR [9]	7.88 ± 0.19	16.7 ± 1.21	5.67 ± 0.36	5.73 ± 1.03	1	10	10	20
LQR [14]	9.6 ± 0.22	17.7 ± 0.8	5.7 ± 0.36	5.79 ± 0.98	4	10	10	30

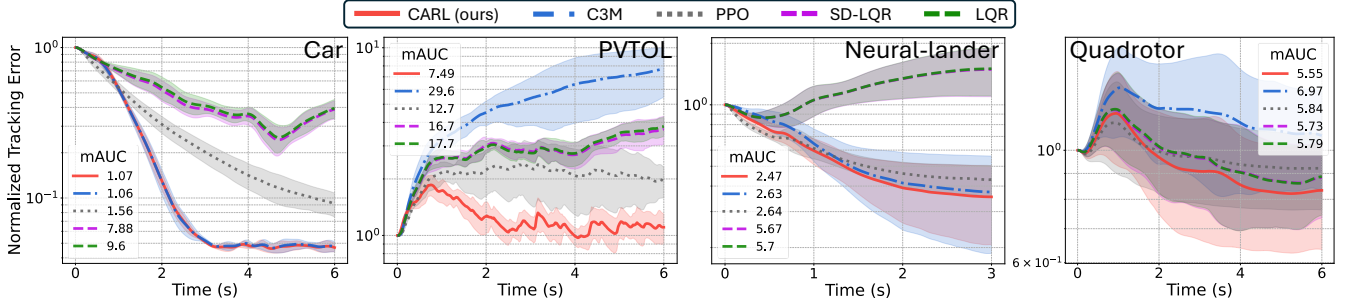


Fig. 5: We plot the mean and 95% confidence intervals of the normalized tracking error, $\|e(t)\|_2$, for CARL and relevant baselines across 5 random seeds.

We evaluated the path-tracking performance of a policy by considering 10 different reference trajectories. For each reference trajectory, we performed rollouts from 10 different initial states near the initial state of that reference trajectory. We then calculated the mean and 95% confidence interval of mAUC over those 100 trajectories.

D. Simulation Results

We report mean and 95% confidence intervals of mAUC and the mean per-step inference time—measured on an Intel i7-1165G7 (2.80GHz) CPU with 16GB of 4266MHz LPDDR4X SDRAM—of each algorithm over 5 random seeds in Table III and depict their corresponding normalized error curves in Figure 5. Additionally, we visualize the path-tracking results in Appendix B (Figure 15). We highlight four key observations.

1) *Performance of CARL:* As shown in Table III, CARL achieves the best performance in the PVTOL, Neural-lander, and Quadrotor environments, and demonstrates comparable performance to C3M in Car (the simplest environment). CARL shows especially strong performance in PVTOL, where it achieves a 41% reduction in mAUC over the next-best baseline. We attribute these results to the cumulative minimization of both transient and steady-state errors in CARL. Additionally, CARL consistently outperforms PPO, highlighting the advantage of our CCM-conditioned reward function. Finally, LQR-based methods perform poorly because LQR relies on linearized dynamics, and neither LQR nor SD-LQR guarantees stabilization of the closed-loop error dynamics for nonlinear systems [9].

2) *Practicality of CARL:* Table III also shows that CARL, C3M, and PPO achieve low (0.1 ms) per-step inference time of controls regardless of system dimensionality, due to the use of feed-forward networks. In comparison, LQR-based methods have nearly $100\times$ longer inference times for solving the Riccati equation at each time step. Moreover,

TABLE IV: **Evaluation results with dynamics models of varying coverage.** Mean and 95% confidence intervals of mAUC over 5 seeds are reported with the best results bolded. PPO is omitted as it does not use a dynamics model.

Method	mAUC (lower is better)			
	Car	PVTOL	Neural-lander	Quadrotor
Control-focused dynamics				
CARL (Ours)	1.05 ± 0.06	7.12 ± 0.75	2.33 ± 0.39	5.42 ± 1.21
C3M [8]	1.06 ± 0.08	19.6 ± 11	2.44 ± 0.39	5.68 ± 1.27
SD-LQR [9]	7.51 ± 2.81	11.0 ± 1.04	5.68 ± 2.31	6.27 ± 1.49
LQR [14]	17.5 ± 7.38	10.8 ± 1.11	5.72 ± 2.13	6.48 ± 1.41
Real-world-focused dynamics				
CARL (Ours)	1.47 ± 0.11	7.87 ± 1.14	2.53 ± 0.83	5.75 ± 1.54
C3M [8]	10.8 ± 1.8	33.2 ± 12.9	3.88 ± 1.13	5.96 ± 1.63
SD-LQR [9]	15.4 ± 5.3	26.7 ± 7.5	9.43 ± 1.0	6.65 ± 1.1
LQR [14]	16.1 ± 6.4	32.5 ± 11.2	9.49 ± 0.87	6.34 ± 0.84

inference time grows with problem dimensionality, indicating that these methods do not scale well. Such scaling is important, given that most robotic systems, such as drones and mobile robots, operate with limited computational capacity in high-dimensional environments.

3) *Robustness of CARL:* We also studied the robustness of CARL to the varying coverage of data used for learning the dynamics model, by considering the control-focused and real-world-focused dynamics models introduced in Section VI-C2. We considered the control-focused dynamics models to understand the impact of emphasizing better coverage of the control space, rather than the state space. We considered the real-world-focused dynamics models to consider a more realistic setting where we cannot collect data uniformly. We do not show results for PPO because it does not use a dynamics model.

As shown in Table IV, CARL outperforms all baselines across all environments when using the control-focused and real-world-focused dynamics models, often by a significant margin. CARL likely outperforms C3M in these settings

TABLE V: **Ablation results for CARL with baseline dynamics model.** Mean and 95% confidence intervals of mAUC over 5 seeds are reported with the results that are better than one in Table III bolded.

Environment	CARL ablation: mAUC (lower is better)		
	Default	Stochastic CMG w/o entropy	Deterministic CMG
Car	1.07 ± 0.05	1.08 ± 0.07	1.08 ± 0.08
PVTOL	7.49 ± 0.84	7.91 ± 1.11	8.79 ± 2.21
Neural-lander	2.47 ± 0.42	2.54 ± 0.58	2.51 ± 0.38
Quadrotor	5.55 ± 1.36	5.29 ± 1.55	5.64 ± 1.45

because it does not directly rely on the approximate dynamics in policy optimization. Specifically, while C3M synthesizes the policy using the approximate CCMs (which are biased metrics due to approximate dynamics), CARL instead incorporates them implicitly through the reward function, where the maximum reward is achieved when the system contracts, regardless of approximation errors in the pretrained dynamics during simulation-based training.

The lack of robustness for LQR-based methods is likely due to their need for linearized dynamics (LQR) and additional approximation of state-dependent coefficients of the dynamics (i.e., $f(x) = A(x)x$) from the approximate dynamics model (SD-LQR), making them more susceptible to model approximation errors.

We also observe that CARL, when evaluated under real-world-focused dynamics, performs better than PPO. This suggests that CARL has the potential to outperform PPO in real-world settings, which is consistent with our robot experimental results, as we discuss in Section VII and Section VIII.

4) *Ablation study of CARL:* Our ablation study compares the performance of CARL—which employs a stochastic, entropy-regularized CMG—against two variants: a deterministic CMG and a stochastic CMG without entropy regularization. Table V reports the mAUC obtained by CARL under each CMG variant. We see that none of the CMG variants yielded better performance (lower mAUC) than the default configuration, except for the stochastic CMG without entropy regularization in Quadrotor. We expect the improved performance of the stochastic CMG without entropy regularization in Quadrotor may be due to tuning of the $\beta_{\mathcal{M}}$ hyperparameter, which regulates the strength of the entropy regularization.

VII. REAL-WORLD EXPERIMENT: TURTLEBOT3 BURGER

We evaluated CARL in three challenging sim-to-real settings. We used the facility depicted in Figure 6a, which has eight Qualisys motion capture cameras to track the position and orientation of the robots. We considered wheeled, aerial, and legged robotic platforms (depicted in Figure 6b). This section focuses on the TurtleBot3 Burger experiment. We discuss the Flapper Drone experiment in Section VIII and the Jethexa experiment in Section IX. A video of all experiments is available at this link.

Our experiment pipeline is shown in Figure 7 and consists of four stages: 1) collecting offline data, 2) conducting system identification, 3) synthesizing a path-tracking policy, and 4) deploying the policy in the real-world. We only considered C3M and PPO as baselines in these real-world experiments,

due to the high per-step inference time of LQR-based algorithms. For CARL, we used the identified system model to create a simulator for pretraining a dynamics model and policy optimization. We gave C3M direct access to the identified system model (i.e., we do not use a pretrained dynamics model). Similar to CARL, PPO used the identified system model to create a simulator for policy optimization.

We did not implement any specific sim-to-real techniques, such as domain randomization [31] or mini-max game for worst-case optimization [32]. We did this to focus our evaluation on the inherent robustness of our considered algorithms to errors from the dynamics mismatch between simulation and the real-world and model approximation errors in formulating CCM conditions. In principle, CARL can be implemented with existing sim-to-real techniques to further reduce the sim-to-real gap. Unless otherwise specified, the experimental setup follows that of our simulated experiments, as discussed in Section VI-C.

A. TurtleBot3 Burger Experiment Details

1) *Offline data collection:* We constructed an offline dataset using 26 reference trajectories. Following the simulated real-world-focused data generation process described in Section VI-C, each reference trajectory was generated by initializing the robot and executing reference controls with added Gaussian noise from that state. The motion capture system recorded the resulting sequence of positions and attitudes of the robot, which we used as reference trajectories. We then used a second-order finite difference method to compute the dynamics $\dot{x}$. We used 20 reference trajectories for training, five for validation, and one for testing (evaluation). Training trajectories were used for system identification and policy optimization, while validation trajectories were used to identify the best-performing checkpoint for deployment. The best-performing policy was then deployed using the test trajectory as a reference trajectory to evaluate path-tracking performance.

2) *System identification:* We modeled the dynamics of the TurtleBot3 Burger as follows:

$$f(x) = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \quad B(x) = \begin{bmatrix} c_1 \cos(\psi) & 0 \\ c_2 \sin(\psi) & 0 \\ 0 & c_3 \end{bmatrix}, \quad (36)$$

where the state is defined as $x := (p_x, p_y, \psi)$, with p_x and p_y denoting the 2D position and ψ denoting the yaw angle. The control input is defined as $u := (v, \omega)$, representing linear and angular velocities, respectively. We identified the empirical coefficients using least-squares regression with 20 training trajectories as follows:

$$c_i = \underset{c \in \mathbb{R}}{\operatorname{argmin}} \sum_{j=1}^{20} \sum_{t=1}^{L_{\max}} \left(\dot{x}_{i,t}^{(j)} - c \cdot \hat{\dot{x}}_{i,t}^{(j)} \right)^2, \quad \forall i \in \{1, 2, 3\}, \quad (37)$$

where j is the trajectory index and t is the time step. Here, $\dot{x}_{i,t}^{(j)}$ denotes the observed dynamics from the motion capture system for the i -th dimension of the state, and $\hat{\dot{x}}_{i,t}^{(j)}$ denotes the corresponding nominal dynamics term (i.e., the i -th row of Equation (36) excluding the empirical coefficient c_i). We show

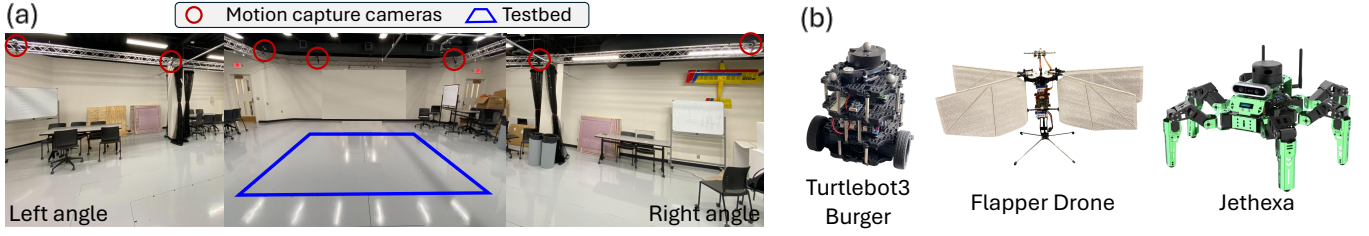


Fig. 6: (a) Our facility for real-world robot experiments, which include a 12×12 ft testbed. Qualisys motion capture cameras are denoted by a red circle. (b) We consider wheeled, aerial, and legged robots.

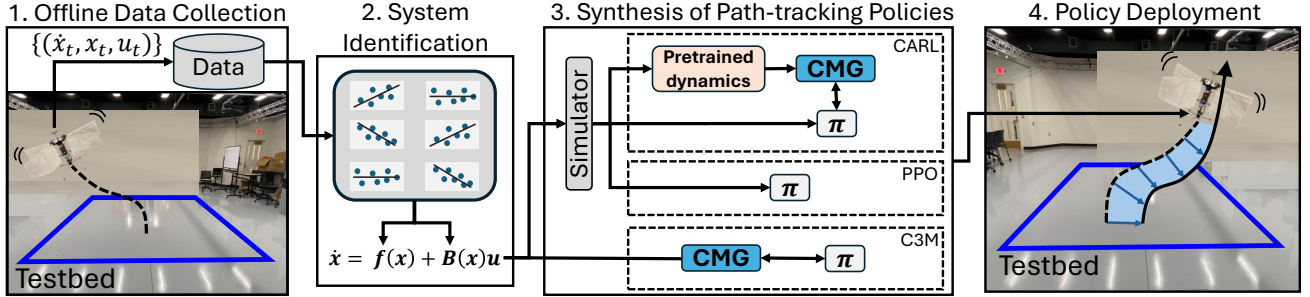


Fig. 7: Our real-world experimental pipeline consists of the following stages: 1) collecting offline data, 2) system identification, 3) policy synthesis, and 4) policy deployment.

TABLE VI: Identified empirical coefficients for the TurtleBot3 Burger dynamics.

Symbol	Value
c_1	0.8183
c_2	0.8179
c_3	0.6767

the identified empirical coefficients in Table VI and visualizations of the least-squares fits in Appendix B (Figure 16a). These coefficients reflect uncertainties such as friction and actuator delays (e.g., motor response and command latency).

3) *Synthesis of path-tracking policies*: For CARL and PPO, we optimized the policy by interacting with a simulator that integrates the identified dynamics from Section VII-A2 using the Euler method. For C3M, we used the same identified dynamics to directly synthesize the contracting policy.

4) *Policy deployment*: We executed policies on the robot at 10 Hz on a centralized computer for 20 seconds. We constructed the policy input (state) by using real-time 3D pose data from the motion capture cameras. We passed the controls output by the policy to the onboard velocity PID controller, which implements corresponding linear and angular velocities.

5) *Evaluation metric*: We evaluated path-tracking performance using mAUC, as defined in Equation (35), by considering the held out test reference trajectory. We randomly initialized five different initial states for the robot near the initial state of this test trajectory, and rollout one trajectory for each initial state. We then calculated the mean and 95% confidence interval of mAUC over those five trajectories.

B. TurtleBot3 Burger Results

Figure 8 shows the reference trajectory (i.e., test trajectory) and executed trajectories from deploying trained policies on a

TurtleBot3 Burger with five different initial states for CARL, C3M, and PPO. Figure 9 shows the corresponding normalized tracking errors for these trajectories, with the mean and 95% confidence intervals. We see that CARL outperforms C3M and PPO, where C3M completely fails to track the path, and PPO exhibits instability during path-tracking with noticeable jittering. In contrast, CARL maintains the most reliable trajectory tracking with the lowest mAUC, demonstrating its robustness in a real-world setting.

VIII. REAL-WORLD EXPERIMENT: FLAPPER DRONE

We also evaluated CARL using the Flapper Drone depicted in Figure 6b, within the same facility shown in Figure 6a. We followed the same experimental pipeline shown in Figure 7.

A. Flapper Drone Experimental Details

1) *Cascaded control approach*: In contrast to the TurtleBot3 robot, the Flapper Drone exhibits challenging instabilities due to its non-rigid wings and nonlinear thrust-voltage dependency, rendering direct control using an on-board velocity-based (low-level) PID controller ineffective. Consequently, we implemented a cascaded control architecture where the policy's low-level commands (e.g., thrust and attitude rate) were first converted into the desired next state (3D position and yaw angle) using the identified dynamics model that we discuss in Section VIII-A3. These computed targets were then tracked by the robot's onboard position-based (high-level) PID controller, which internally generates low-level commands for the velocity-based PID loop. Our experiments show this cascaded architecture to be much more reliable than direct control for all algorithms considered.

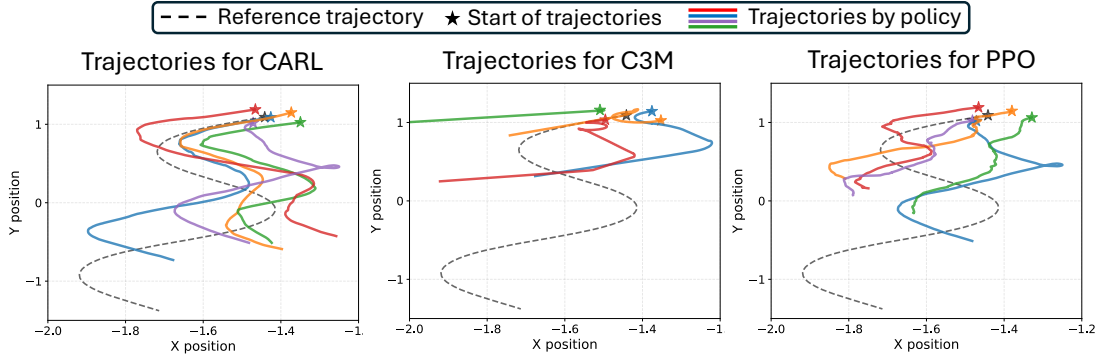


Fig. 8: We demonstrated our algorithm (CARL) on a TurtleBot3 Burger along with key baselines. The robot was initialized at five distinct initial positions to track the same reference trajectory across all algorithms.

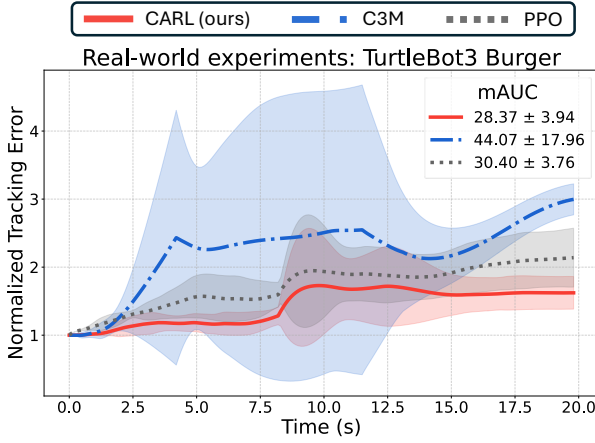


Fig. 9: Mean and 95% confidence intervals for the normalized tracking error over 5 seeds from real-world experiments. The higher mAUC observed, compared to that in Table III, is attributed to the longer horizon length and increased uncertainty from the sim-to-real transfer.

2) *Offline data collection:* Since we employ the position-based PID controller for the Flapper Drone, we collected offline data by first generating position-based reference trajectories and executing a position-based PID controller to track them. The position-based reference trajectories are as follows:

$$\begin{aligned} p_x(t) &:= p_x(0) + (p_x(T_{\max}) - p_x(0)) \frac{t}{T_{\max}} + \varsigma_x \sin(2\pi \varrho_x t), \\ p_y(t) &:= p_y(0) + (p_y(T_{\max}) - p_y(0)) \frac{t}{T_{\max}} + \varsigma_y \sin(2\pi \varrho_y t), \\ p_z(t) &:= p_z(0) + (p_z(T_{\max}) - p_z(0)) \frac{t}{T_{\max}}, \end{aligned} \quad (38)$$

where $p_x(0), p_y(0), p_z(0)$ and $p_x(T_{\max}), p_y(T_{\max}), p_z(T_{\max})$ represent the initial and final 3D positions, $T_{\max}$ is the maximum flight time, and ς and ϱ denote the amplitude and frequency of the sinusoidal function, respectively. Note that we apply the sinusoidal perturbation to only planar positions p_x and p_y . The initial planar positions were sampled uniformly as $p_x, p_y \sim \text{Unif}([-0.3, 0.3])$, while the vertical position was fixed at the robot's nominal height. Target positions were generated by applying a displacement vector Δp to the initial positions, such that $p(T_{\max}) = p(0) + \Delta p$. The displacement components were sampled as $\Delta p_x, \Delta p_y \sim \text{Unif}([-0.3, 0.3])$

TABLE VII: Identified empirical coefficients for the Flapper Drone dynamics.

Symbol	Value	Symbol	Value	Symbol	Value
c_1	1.0	c_5	1.6567	c_8	1.0
c_2	1.0	c_6	6.5546	c_9	1.0
c_3	1.0	b_6	3.1939	c_{10}	1.0
c_4	1.6321	c_7	1.0		

and $\Delta p_z \sim \text{Unif}([1.0, 1.5])$. Additionally, we selected small amplitudes $\varsigma_x, \varsigma_y \sim \text{Unif}([0.05, 1])$ and low frequencies $\varrho \sim \text{Unif}([0.01, 0.05])$ to ensure the reference trajectories remain trackable despite high dynamic uncertainties of the robot, and we fixed the yaw angle to zero (i.e., $\psi = 0$).

Following the setup in Section VII-A, we computed the robot dynamics and collected 26 reference trajectories, 20 of which were used for training, five for validation, and one for testing (evaluation).

3) *System identification:* We modeled the dynamics of the Flapper Drone using the 10D quadrotor model described in Section VI-A with empirical coefficients as follows:

$$\begin{aligned} f(x) = & \begin{bmatrix} c_1 v_x \\ c_2 v_y \\ c_3 v_z \\ c_4 \tilde{f} \sin(\theta) \\ -c_5 \tilde{f} \cos(\theta) \sin(\phi) \\ c_6 \tilde{f} \cos(\theta) \cos(\phi) - b_6 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad B(x) = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ c_7 & 0 & 0 & 0 \\ 0 & c_8 & 0 & 0 \\ 0 & 0 & c_9 & 0 \\ 0 & 0 & 0 & c_{10} \end{bmatrix}. \end{aligned} \quad (39)$$

We identified the empirical coefficients using linear least-squares regression. For indices $i \in \{1, 2, 3, 4, 5, 7, 8, 9, 10\}$ (except 6-th index), we use the standard formulation as in Equation (37). However, for the 6-th index where there exists gravitational effect, we include a bias term b_6 and solve affine least-squares regression:

$$c_6, b_6 = \underset{c, b \in \mathbb{R}}{\operatorname{argmin}} \sum_{j=1}^{20} \sum_{t=1}^{L_{\max}} \left(\dot{x}_{6,t}^{(j)} - (c \cdot \dot{x}_{6,t}^{(j)} + b) \right)^2, \quad (40)$$

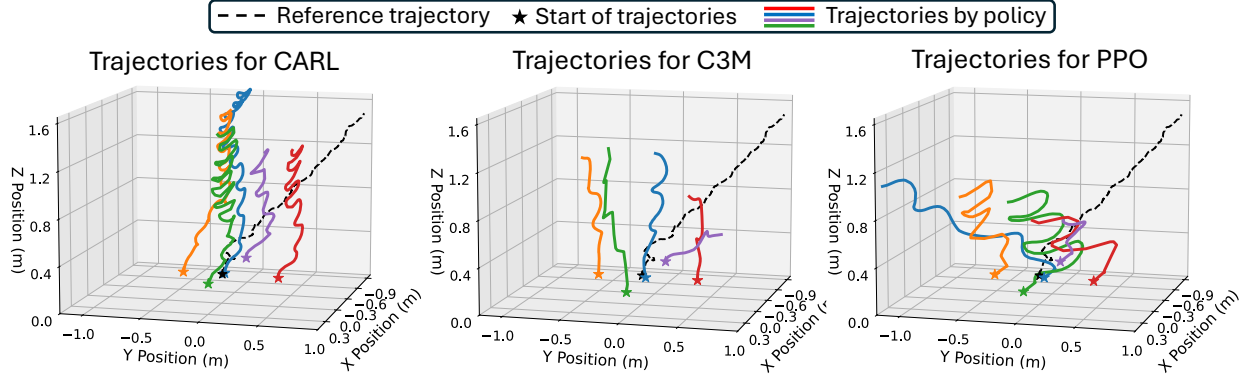


Fig. 10: We demonstrated our algorithm (CARL) on the Flapper Drone along with key baselines. The robot was initialized at five distinct starting positions to track the same reference trajectory across all algorithms.

where j is the trajectory index and t is the time step. Here, $\hat{x}_{i,t}^{(j)}$ denotes the observed dynamics from the motion capture system (for position and orientation) or computed via finite differences (for the rest of the state indices). $\hat{\hat{x}}_{i,t}^{(j)}$ denotes the corresponding nominal dynamics term (i.e., the i -th row of Equation (39) excluding the empirical coefficients). We show the identified empirical coefficients in Table VII and visualizations of the least-squares fits in Appendix B (Figure 16b). These empirical coefficients capture the unique dynamical properties of the Flapper Drone, where the acceleration terms are weighted by coefficients to represent the relationship between the flapping wing motor output and the resulting thrust generation.

Note that the empirical coefficient b_6 should correspond to gravitational acceleration, yet it was identified as 3.1939. This discrepancy arises because the Flapper Drone operates on a non-physical hardware thrust scale (10,001–60,000) and uses a normalized thrust in its state. However, the coefficient is functionally correct as it calibrates the model's hover equilibrium (i.e., $\dot{v}_z = 0$) to a thrust of $\hat{f} = 0.487$, which matches the robot's actual hover thrust.

4) *Synthesis of path-tracking policies*: We adopted the policy synthesis procedure detailed in Section VII-A3.

5) *Policy deployment*: We executed the policy on a centralized computer at 20 Hz for 30 seconds, while internal sensors and motion capture operated at 100 Hz, thereby avoiding potential sampling instabilities. We constructed the policy input (state) by using real-time 3D pose data obtained from the motion capture cameras, velocity estimates from the onboard Kalman filter, and a thrust obtained via direct integration ($\hat{f}_{t+1} \leftarrow \hat{f}_t + \hat{f} \cdot \Delta t$), where $\hat{f}$ is part of the controls output by the policy. We integrated the thrust because we observed a significant latency in the robot's thrust telemetry. Finally, we passed the controls output by the policy to the identified system model to generate the target next 3D position $\hat{p}_x, \hat{p}_y, \hat{p}_z$ and yaw angle $\hat{\psi}$, which were then passed to the on-board position-based PID controller for execution (see Section VIII-A1).

6) *Evaluation metric*: For these experiments, we slightly revise the mAUC evaluation metric used in Section VII-A5 to penalize early termination caused by safety violations, where the robot flew out of a pre-defined safety envelope (which

triggered an immediate abort to prevent hardware damage). Since early termination represents a critical control failure, we impose an additional penalty for early termination by scaling mAUC by the ratio $\frac{L_{\max}}{L}$. The resulting metric, denoted as $m^2\text{AUC}$, is defined as follows:

$$m^2\text{AUC} := \frac{L_{\max}}{L} \cdot \underbrace{\frac{L_{\max}}{L} \sum_{k=0}^L \|e(t_k)\|_2}_{\text{mAUC}}. \quad (41)$$

Additionally, we provide a linear error trend (τ^*) and path-tracking completion (ι) for a comprehensive assessment under early termination, where τ and ι are defined as follows:

$$\tau^* := \underset{\tau}{\operatorname{argmin}} \sum_{k=0}^L (\|e(t_k)\|_2 - (\tau t_k + 1))^2, \quad (42)$$

$$\iota := \frac{L}{L_{\max}} \times 100\%. \quad (43)$$

The bias term in finding τ^* is constrained to 1 to strictly measure the linear error trend, reflecting the fact that the initial normalized tracking error is always 1.

B. Flapper Drone Results

Figure 10 shows the reference trajectory (i.e., test trajectory) and executed trajectories from deploying trained policies on the Flapper Drone with five different initial states for CARL, C3M, and PPO. Figure 11a shows corresponding normalized tracking errors with linear error trend and Figure 11b plots the $m^2\text{AUC}$ and path-tracking completion achieved by each algorithm. While the sim-to-real gap introduces large tracking errors across all algorithms, CARL demonstrates superior performance and robustness relative to baselines. For example, Figure 11 shows that CARL outperforms the baselines across all evaluated metrics: $m^2\text{AUC}$, linear error trend, and tracking completion. The short path-tracking completions for C3M and PPO are due to erroneous control inputs that drive the robot outside the safety region. Conversely, CARL produces controls that better regulate the Flapper Drone to the reference trajectory, mitigating the risk of early termination and achieving the best path-tracking performance.

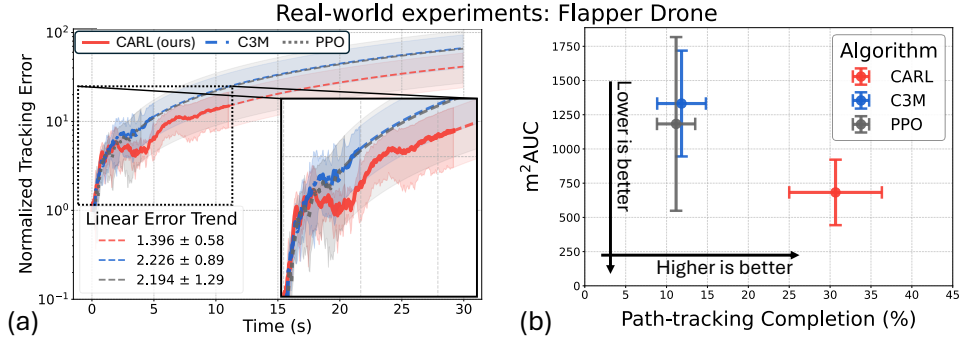


Fig. 11: (a) We plot the mean and 95% confidence intervals of the normalized tracking error over 5 seeds from Flapper experiments. Additionally, (b) we plot the corresponding mean and 95% confidence intervals of m^2AUC and path-tracking completion, where the bottom-right corner denotes the most desirable performance (high completion and low error). The higher m^2AUC observed, compared to that in Table III, is attributed to the longer horizon length and increased uncertainty from the sim-to-real transfer.

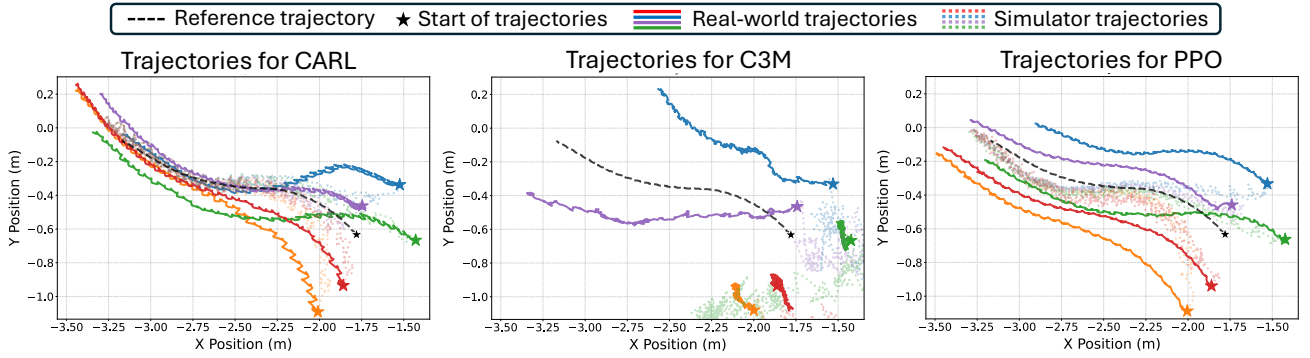


Fig. 12: We demonstrated our algorithm (CARL) on the Jethexa along with key baselines. The robot was initialized at five distinct starting positions to track the same reference trajectory across all algorithms.

IX. REAL-WORLD EXPERIMENT: JETHEXA

To scale beyond low-dimensional mobile robots such as TurtleBot3 Burger and Flapper Drone, we evaluated CARL on the Jethexa hexapod—which features complex kinematic coupling between state elements—depicted in Figure 6b, within the same facility shown in Figure 6a. We followed the same experimental pipeline shown in Figure 7.

A. Jethexa Experimental Details

1) *Offline data collection:* The Jethexa platform provides an inverse kinematics solver that translates given velocity commands into a desired locomotion. We used this approach to generate reference trajectories by sampling a forward linear velocity $v_x \sim \text{Unif}(0.02, 0.05)$, while holding the lateral velocity v_y zero. Additionally, we included an angular velocity using a sinusoidal function as $\omega = \varsigma \sin(2\pi\rho)$ where we sampled $\varsigma \sim \text{Unif}(0.02, 0.05)$ and a frequency of $\rho := 2\pi/T_{\max}$, ensuring it completes exactly one full cycle while generating a diverse set of reference trajectories. We use the motion capture system in our facility (Figure 6a) to record the robot's base position and attitude, while joint angles are read directly from the robot. Similarly, we use the second-order finite difference method to compute the dynamics $\dot{x}$.

Following the setup in Section VII-A, we computed the robot dynamics and collected 26 reference trajectories, 20 of which were used for training, five for validation, and one for testing (evaluation).

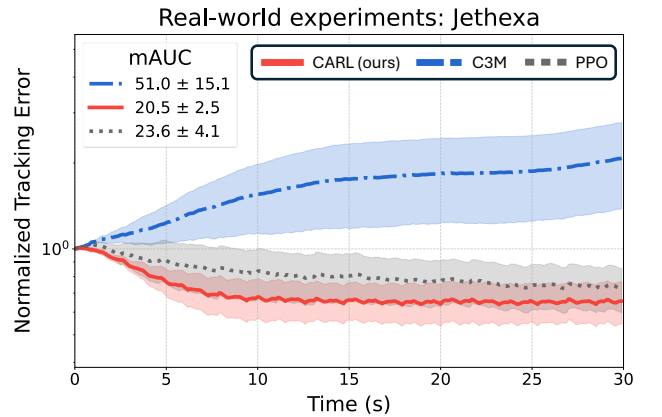


Fig. 13: Mean and 95% confidence intervals for the mAUC from simulation experiments. The higher mAUC observed, compared to that in Table III, is attributed to the longer horizon length and increased uncertainty from the sim-to-real transfer.

2) *System identification:* We modeled the dynamics of the Jethexa as follows:

$$f(x) = \mathbf{0}_{23 \times 1}, \quad B(x) = \begin{bmatrix} \mathcal{J}_\omega(x) \\ I_{18 \times 18} \end{bmatrix}, \quad (44)$$

where the state is defined as $x := (p_x, p_y, \phi, \theta, \psi, \mathbf{q}_{18})$, with p_x and p_y denoting the 2D position, ϕ, θ , and ψ are the attitude, and $\mathbf{q}_{18}$ denotes the 18 joint angles. Specifically, the model

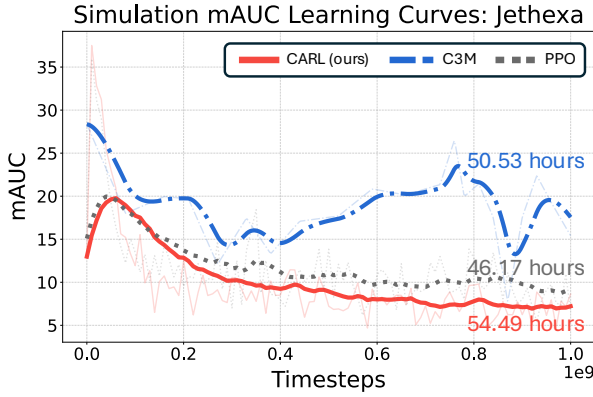


Fig. 14: Learning curves of the mAUC for each algorithm during Jethexa simulation experiments. The total wall-clock time for training is denoted.

consists of six legs, each featuring three joints (the coxa, femur, and tibia), resulting in 18 joint angles. The control input is defined as $u := \dot{\mathbf{q}}_{18}$, representing angular velocities of each joint. The base dynamics $\mathcal{J}_{\varpi}(x)$ is produced for each state x by a feed-forward neural network with parameters ϖ trained on training data (see Section IX-A1) as described in Section IV-A. ϖ was represented by hidden layers of size (64, 64) and Tanh activation functions. We visualize the prediction results of $\mathcal{J}_{\varpi}(x)$ on the dynamics of the test trajectories and the corresponding mean-squared error in Appendix B (Figure 16c).

3) *Synthesis of path-tracking policies:* We adopted the policy synthesis procedure detailed in Section VII-A3.

4) *Policy deployment:* We executed the policy on a centralized computer at 20 Hz for 30 seconds, while internal sensors operated at 50 Hz and motion capture at 100 Hz. We constructed the policy input (state) using real-time data obtained from motion capture cameras and joint angles from the onboard sensor. We integrated the control outputs (i.e., joint angle velocities) by the policy as $\mathbf{q}_{18} \leftarrow \mathbf{q}_{18} + \dot{\mathbf{q}}_{18}\Delta t$ and clipped to a safe operational range defined by 10% expansion of the joint angle range found in the training data. Finally, the clipped joint angle target is then commanded to the onboard PID controllers.

5) *Evaluation metric:* We adopted the same evaluation procedure established for the TurtleBot3 Burger in Section VII-A5.

B. Jethexa Results

Figure 12 shows the reference trajectory (i.e., test trajectory) and executed trajectories from deploying trained policies on the Jethexa with five different initial states for CARL, C3M, and PPO. We illustrate the sim-to-real gap by including open-loop predicted trajectories (shown as dotted lines) that represent the policy’s expected path from each initial state under identified dynamics in Equation (44). Figure 13 shows the corresponding normalized tracking errors for these trajectories, with the mean and 95% confidence intervals. The mAUC learning curves and corresponding total wall-clock time for training of each algorithm are also depicted in Figure 14.

We observe that CARL outperforms both C3M and PPO in real-world deployment and simulation experiment as depicted in Figure 13 and Figure 14, respectively. As seen in Figure 12, CARL adopts a slightly more aggressive control strategy to minimize cumulative tracking error in a contracting manner to effectively track the reference trajectory. In contrast, C3M fails to maintain stable tracking—with several trials diverging significantly—while PPO, despite its smoother control profile, exhibits noticeable path-tracking error compared to CARL. Finally, we see that CARL shows an increase in training wall-clock time of only 8% relative to C3M and 18% relative to PPO.

We note that both CARL and the baselines achieve better real-world tracking performance on the Jethexa compared to the TurtleBot3 Burger and Flapper Drone. Two potential reasons for this improvement are that the Jethexa is more robust to uncertainties, like friction and latency dynamics, which reduces the estimation error due to data scarcity in system identification, and that we used a neural network to model its dynamics, rather than least-squares.

X. CONCLUSIONS

We propose a contraction-aware RL algorithm, CARL, for robust path-tracking in high-dimensional nonlinear systems under approximate dynamics. Our key idea is to simultaneously learn a CMG and a policy, where the policy optimizes a CCM-conditioned reward function defined by CCMs generated by the CMG. Our resulting policy achieves stability and long-term optimality, as well as robustness to dynamics approximation errors. While our results use PPO for RL policy optimization, in principle, CARL can be implemented with any RL algorithm. We empirically demonstrate the benefits of CARL relative to baselines through extensive simulated and real-world experiments. We also theoretically analyze the stability of CARL and its robustness under approximate dynamics.

One limitation of CARL is its reliance on a simulator for RL training. However, recent progress in robotics has resulted in many high-fidelity simulators, such as NVIDIA Isaac Sim and Genesis, which provide fast, realistic, and easily-integrable environments for training a policy.

We also outline several promising directions for future work. First is extending CARL to the offline RL setting, which would address our reliance on a simulator. Second is considering robust RL formulations—such as domain randomization or minimax worst-case optimization—to further improve the sim-to-real gaps shown in our robot experiments and compare with additional baselines (e.g., well-tuned classical controllers). These additions should better clarify the practical significance of our work. Other directions include jointly training all models (dynamics, CMG, and policy).

ACKNOWLEDGMENTS

Listed in alphabetical order, the authors would like to thank Rihan D’silva, Inyoung Kim, and Hamid Osooli for their insightful discussions and valuable assistance with the real-world robot experiments.

This paper will appear in the IEEE Transactions on Robotics. © 2026 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.

APPENDIX A OMITTED PROOFS

Proof of Lemma 1: Given $\tilde{C}(s) = 1 - R(s)$, the objective based on the reward value function of any policy π is given as follows:

$$J_R(\pi) = \mathbb{E}_{s_0} \left[\mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t (1 - \tilde{C}(s_t)) \mid s_0 \right] \right] \quad (45)$$

$$= \frac{1}{1 - \gamma} - J_{\tilde{C}}(\pi), \quad (46)$$

where we used geometric series as $\gamma < 1$ ensures the series converge. Given that the term $(1 - \gamma)^{-1}$ is constant, and assuming consistent tie-breaking rules, the optimization problem simplifies to

$$\pi^* = \operatorname{argmax}_{\pi} \left(\frac{1}{1 - \gamma} - J_{\tilde{C}}(\pi) \right) \quad (47)$$

$$= \operatorname{argmax}_{\pi} (-J_{\tilde{C}}(\pi)) = \operatorname{argmin}_{\pi} J_{\tilde{C}}(\pi). \quad (48)$$

□

Proof of Lemma 2: Let $\tau_k := 2\lambda k \Delta t$ for notational simplicity. By the strict monotonicity of $\tilde{C}$ and the original contraction bound $C(s_{t+k}) \leq C(s_t)e^{-\tau_k}$, we have:

$$\tilde{C}(s_{t+k}) \leq \frac{C(s_t)e^{-\tau_k}}{1 + C(s_t)e^{-\tau_k}} \quad (49)$$

$$= \tilde{C}(s_t) \left(\frac{1 + C(s_t)}{1 + C(s_t)e^{-\tau_k}} \right) e^{-\tau_k} \quad (50)$$

$$= \tilde{C}(s_t) \exp \left(-\tau_k + \ln \frac{1 + C(s_t)}{1 + C(s_t)e^{-\tau_k}} \right). \quad (51)$$

Defining the k -th step contraction rate $\hat{\lambda}_k$ such that the exponent in Equation (51) equals $-2\hat{\lambda}_k k \Delta t$, we obtain:

$$\hat{\lambda}_k = \lambda \left(1 - \frac{1}{\tau_k} \ln \frac{1 + C(s_t)}{1 + C(s_t)e^{-\tau_k}} \right). \quad (52)$$

Because $e^{-\tau_k} < 1$, it trivially holds that

$$0 < \ln \frac{1 + C(s_t)}{1 + C(s_t)e^{-\tau_k}} < \tau_k, \quad (53)$$

ensuring $\hat{\lambda}_k \in (0, \lambda)$. Let $\tilde{\lambda} := \inf_{k \in \mathbb{N}} \hat{\lambda}_k > 0$. Then, there exists a rate $\tilde{\lambda} \in (0, \lambda)$ such that π_c contracts with respect to $\tilde{C}$ as follows:

$$\tilde{C}(s_{t+k}) \leq \tilde{C}(s_t)e^{-2\tilde{\lambda}k\Delta t}, \quad \forall k \in \mathbb{N}. \quad (54)$$

□

We establish the following lemma that we extensively use to prove the main theorems.

Lemma 3: (Uniformly Bounded Cumulative Cost of an optimal Policy). Suppose there exists at least one policy π_c that contracts with respect to C with rate $\lambda > 0$ as in Assumption 1. Let the cost function and corresponding cumulative cost be defined as in Equation (17) and Equation (18), respectively. Then, the cost value function of a deterministic optimal policy π^* satisfying Equation (16) is uniformly bounded as follows:

$$V_C^{\pi^*}(s_t) \leq V_C^{\pi_c}(s_t) = \sum_{k=0}^{\infty} \gamma^k \tilde{C}(s_{t+k}) \leq \frac{\tilde{C}(s_t)}{1 - \gamma e^{-2\tilde{\lambda}\Delta t}}, \quad (55)$$

where $\tilde{\lambda}$ is the contraction rate induced by π_c with respect to the cost function $\tilde{C}$ as in Lemma 2.

Proof of Lemma 3: By Lemma 2, there exists a rate $\tilde{\lambda} \in (0, \lambda)$ such that π_c contracts with respect to $\tilde{C}$. Thus, using the optimality of π^* and the fact that the cost value function of π_c is uniformly bounded using geometric series, we get the following relationship:

$$V_C^{\pi^*}(s_t) \leq V_C^{\pi_c}(s_t) = \sum_{k=0}^{\infty} \gamma^k \tilde{C}(s_{t+k}) \leq \frac{\tilde{C}(s_t)}{1 - \gamma e^{-2\tilde{\lambda}\Delta t}}. \quad (56)$$

□

Proof of Theorem 1: We start from the Bellman equation of the optimal policy π^* . Given Lemma 1, we write the equation in terms of cost using $V_C^{\pi^*}(s_t)$:

$$V_C^{\pi^*}(s_t) = \tilde{C}(s_t) + \gamma V_C^{\pi^*}(s_{t+1}), \quad (57)$$

where the expectation operator is omitted from the RHS, as we focus on deterministic policy and dynamics. Rearranging Equation (57) isolates the difference in the cost value function:

$$V_C^{\pi^*}(s_{t+1}) - V_C^{\pi^*}(s_t) = \frac{1}{\gamma} \left((1 - \gamma)V_C^{\pi^*}(s_t) - \tilde{C}(s_t) \right). \quad (58)$$

For the value function to strictly decrease, the term inside the parentheses on the RHS must be negative, which yields the following condition:

$$V_C^{\pi^*}(s_t) < \frac{\tilde{C}(s_t)}{1 - \gamma}. \quad (59)$$

To prove the existence of $\gamma \in (0, 1)$ that satisfies Equation (59), we bound the LHS of Equation (59) by a finite measure while we let the RHS approach infinity as $\gamma \rightarrow 1$.

Using Lemma 3, we substitute the upper bound of the cost value function of the contracting policy π_c to the LHS of Equation (59) as follows:

$$\frac{\tilde{C}(s_t)}{1 - \gamma e^{-2\tilde{\lambda}\Delta t}} < \frac{\tilde{C}(s_t)}{1 - \gamma} \implies e^{-2\tilde{\lambda}\Delta t} < 1, \quad (60)$$

which proves the cost value function of π^* is a strictly decreasing sequence (i.e., $\{V_C^{\pi^*}(s_t)\}_{t=0}^{\infty}$) for $\gamma \in (0, 1)$.

Given that the value function is bounded below by 0, it converges to some limit $L \geq 0$ by the monotone convergence theorem [33]. Taking $t \rightarrow \infty$ in the Bellman equation from Equation (57) gives

$$\lim_{t \rightarrow \infty} \tilde{C}(s_t) = (1 - \gamma)L. \quad (61)$$

Again, applying Lemma 3 to Equation (61) yields

$$L \leq (1 - \gamma)L / (1 - \gamma e^{-2\bar{\lambda}\Delta t}). \quad (62)$$

Suppose $L > 0$. Dividing both sides by L and simplifying yields $e^{-2\bar{\lambda}\Delta t} \geq 1$, which contradicts the fact that $\bar{\lambda} > 0$ and $\Delta t > 0$. Hence $L = 0$, implying incremental asymptotic stability:

$$\lim_{k \rightarrow \infty} \|\delta x(t_k)\|_M^2 = 0, \quad \forall \delta x(t_0) \in \mathbb{R}^n. \quad (63)$$

□

Proof of Theorem 2: By factoring the rearranged Bellman equation (Equation (57)) with respect to $V_{\tilde{C}}^{\pi^*}(s_t)$, we have

$$V_{\tilde{C}}^{\pi^*}(s_{t+1}) = V_{\tilde{C}}^{\pi^*}(s_t) \left[\frac{1}{\gamma} \left(1 - \frac{\tilde{C}(s_t)}{V_{\tilde{C}}^{\pi^*}(s_t)} \right) \right] = V_{\tilde{C}}^{\pi^*}(s_t) \rho_t, \quad (64)$$

where ρ_t is a contraction factor (i.e., $\rho_t < 1$, $\forall t$) by using $\gamma \in (0, 1)$ (see the proof of Theorem 1).

Given that the value function is always lower bounded by its instantaneous cost and upper bounded as shown in Lemma 3, we establish the following inequality based on Equation (64):

$$\tilde{C}(s_{t+1}) \leq V_{\tilde{C}}^{\pi^*}(s_{t+1}) = V_{\tilde{C}}^{\pi^*}(s_t) \rho_t \leq \tilde{C}(s_t) \frac{\rho_t}{1 - \gamma e^{-2\bar{\lambda}\Delta t}}. \quad (65)$$

Let us define $\hat{\rho}_t := \frac{\rho_t}{1 - \gamma e^{-2\bar{\lambda}\Delta t}}$ and $\bar{\rho} := \sup_t \hat{\rho}_t$. If $\bar{\rho} < 1$, then a deterministic optimal policy π^* exhibits incremental exponential stability as follows:

$$\tilde{C}(s_{t+1}) \leq \tilde{C}(s_t) \hat{\rho}_t \leq \tilde{C}(s_0) \left(\prod_{i=0}^t \hat{\rho}_i \right) \leq \tilde{C}(s_0) \bar{\rho}^{t+1}. \quad (66)$$

Since $\tilde{C}(s_t)$ is strictly monotonic, the contraction of $\tilde{C}(s_t)$ in Equation (66) implies contraction with respect to $C(s_t)$ for some contraction factor. Hence, the following holds:

$$\frac{C(s_{t+1})}{1 + C(s_{t+1})} \leq \frac{C(s_t)}{1 + C(s_t)} \bar{\rho}, \quad \forall t \quad (67)$$

$$\implies C(s_{t+1}) \leq \frac{C(s_t)(1 + C(s_{t+1}))}{1 + C(s_t)} \bar{\rho} \leq C(s_t) \bar{\rho}, \quad \forall t, \quad (68)$$

which directly implies incremental exponential stability as follows:

$$\|\delta x(t_k)\|_M^2 \leq \|\delta x(t_0)\|_M^2 \bar{\rho}^k, \quad \forall k \in \mathbb{N}. \quad (69)$$

Next, we show when $\bar{\rho} < 1$ is true using the following condition:

$$\hat{\rho}_t = \frac{1}{\gamma(1 - \gamma e^{-2\bar{\lambda}\Delta t})} \left(1 - \frac{\tilde{C}(s_t)}{V_{\tilde{C}}^{\pi^*}(s_t)} \right) < 1, \quad \forall t, \quad (70)$$

which simplifies to showing

$$1 - \frac{\tilde{C}(s_t)}{V_{\tilde{C}}^{\pi^*}(s_t)} < \gamma(1 - \gamma e^{-2\bar{\lambda}\Delta t}), \quad \forall t. \quad (71)$$

Applying Lemma 3 to Equation (71) simplifies the condition for incremental exponential stability as follows:

$$\gamma e^{-2\bar{\lambda}\Delta t} < \gamma(1 - \gamma e^{-2\bar{\lambda}\Delta t}) \implies e^{-2\bar{\lambda}\Delta t} < \frac{1}{1 + \gamma}. \quad (72)$$

Rewriting Equation (72) in terms of γ , we define the threshold on the discount factor for incremental exponential stability as follows:

$$\bar{\gamma} := e^{2\bar{\lambda}\Delta t} - 1 > 0. \quad (73)$$

Hence, for a discount factor $\gamma < \min(\bar{\gamma}, 1)$, there exists a contraction factor $\bar{\rho} \in (0, 1)$ such that a deterministic optimal policy π^* satisfying Equation (16) renders the closed-loop system incrementally exponentially stable. □

Proof of Theorem 3: As shown in [27] for Lyapunov functions, the contracting policy π_c yields

$$\mathbb{P} \left[\sqrt{C(s_k)} \leq \sqrt{C(s_0)} e^{-\lambda k \Delta t} + \frac{\sqrt{m} \mathbf{Q}_{1-\varepsilon}}{\lambda} \right] \geq 1 - \varepsilon. \quad (74)$$

Since π_c is deterministic and we have $\sqrt{\sum_{k=0}^{\kappa-1} C(s_k)} \leq \sum_{k=0}^{\kappa-1} \sqrt{C(s_k)}$ for any nonnegative sequence $\{C(s_k)\}$, summing the event of Equation (74) over k and applying geometric series gives

$$\sqrt{\mathbb{E}_{\pi_c} \left[\sum_{k=0}^{\kappa-1} C(s_k) \right]} \leq \underbrace{\frac{(1 - e^{-\lambda \kappa \Delta t}) \sqrt{C(s_0)}}{1 - e^{-\lambda \Delta t}} + \frac{\sqrt{m} \mathbf{Q}_{1-\varepsilon} \kappa}{\lambda}}_{:= \mathbf{UB}}. \quad (75)$$

Thus, assuming π^* achieves lower cumulative tracking error than π_c , taking limit $\kappa \rightarrow \infty$ gives the following bound for the deterministic optimal policy π^* :

$$\mathbb{P} \left[\lim_{\kappa \rightarrow \infty} \frac{\sqrt{\mathbb{E}_{\pi^*} \left[\sum_{k=0}^{\kappa-1} C(s_k) \right]}}{\kappa} \leq \frac{\mathbf{UB}}{\kappa} \right] \geq 1 - \varepsilon. \quad (76)$$

Since π^* is deterministic, this implies Equation (27). □

APPENDIX B ADDITIONAL RESULTS

We depict path-tracking results of CARL and baseline algorithms in Figure 15 for the simulated environments. We plot the system identification results for the TurtleBot3 Burger, Flapper Drone, and Jethexa in Figure 16.

REFERENCES

- [1] I. R. Manchester and J.-J. E. Slotine, "Control contraction metrics: Convex and intrinsic criteria for nonlinear feedback design," *IEEE Transactions on Automatic Control*, vol. 62, no. 6, pp. 3046–3053, 2017.
- [2] W. Lohmiller and J.-J. E. Slotine, "On contraction analysis for non-linear systems," *Automatica*, vol. 34, no. 6, pp. 683–696, 1998. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0005109898000193>
- [3] S. Singh, A. Majumdar, J.-J. Slotine, and M. Pavone, "Robust online motion planning via contraction theory and convex optimization," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2017, pp. 5883–5890.
- [4] H. Tsukamoto and S.-J. Chung, "Robust controller design for stochastic nonlinear systems via convex optimization," *IEEE Transactions on Automatic Control*, vol. 66, no. 10, pp. 4731–4746, 2020.
- [5] —, "Neural contraction metrics for robust estimation and control: A convex optimization approach," *IEEE Control Systems Letters*, vol. 5, no. 1, pp. 211–216, 2020.
- [6] S. Singh, V. Sindhwani, J.-J. E. Slotine, and M. Pavone, "Learning Stabilizable Dynamical Systems via Control Contraction Metrics," Nov. 2018, arXiv:1808.00113 [cs]. [Online]. Available: <http://arxiv.org/abs/1808.00113>

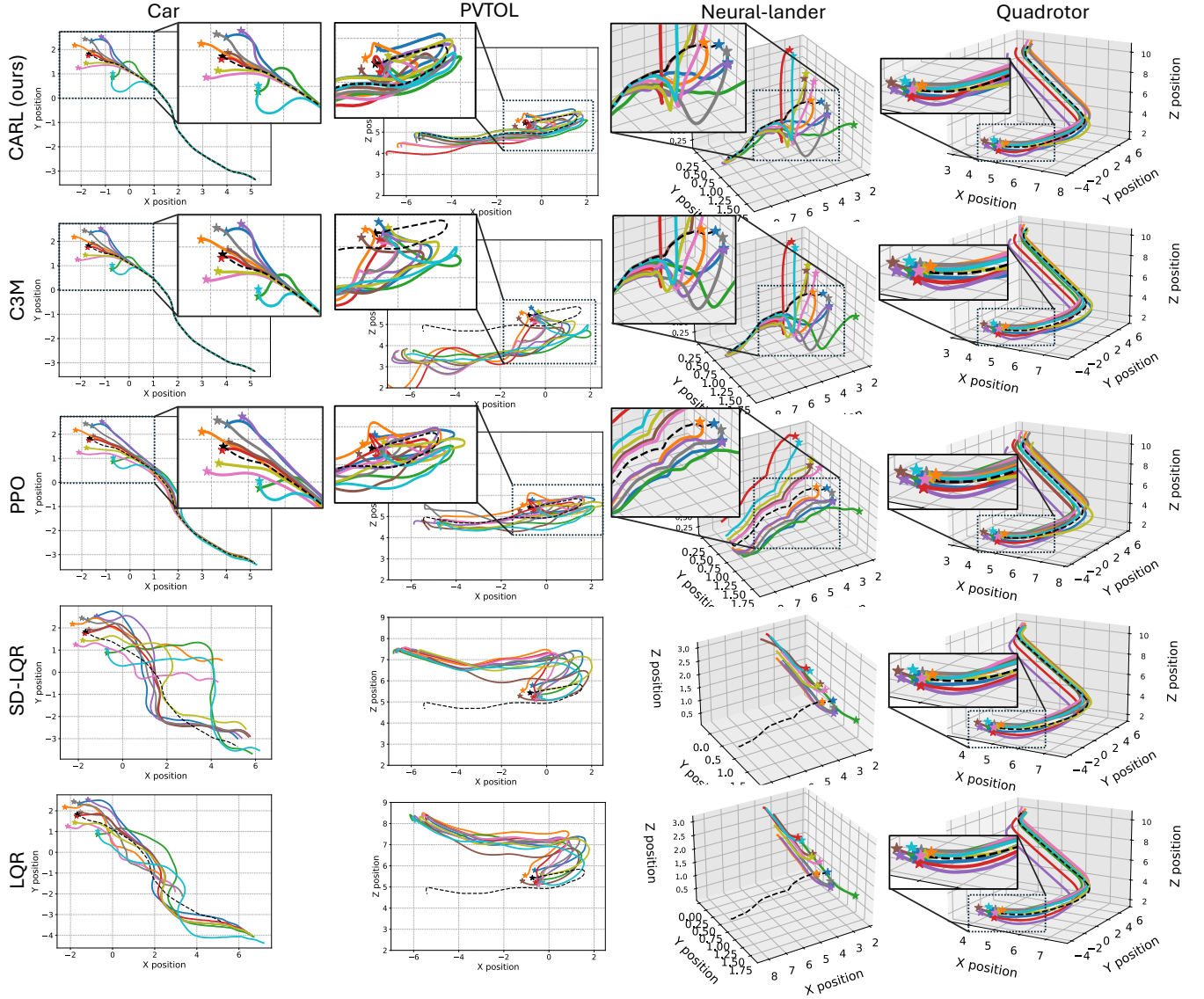


Fig. 15: We plot simulated trajectories of final policies for each algorithm, including magnified insets to highlight stabilizing behaviors. We omit these insets for the LQR-based methods in the CAR, PVTOL, and Neural-lander environments, as their divergence is already clearly visible at the standard scale.

- [7] C. Dawson, S. Gao, and C. Fan, “Safe control with learned certificates: A survey of neural lyapunov, barrier, and contraction methods for robotics and control,” *IEEE Transactions on Robotics*, vol. 39, no. 3, pp. 1749–1767, 2023.
- [8] D. Sun, S. Jha, and C. Fan, “Learning certified control using contraction metric,” in *conference on Robot Learning*. PMLR, 2021, pp. 1519–1539.
- [9] S. M. Richards, J.-J. Slotine, N. Azizan, and M. Pavone, “Learning control-oriented dynamical structure from data,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 29 051–29 062.
- [10] B. Song, J.-J. Slotine, and Q.-C. Pham, “Stable modular control via contraction theory for reinforcement learning,” in *6th Annual Learning for Dynamics & Control Conference*. PMLR, 2024, pp. 1136–1148.
- [11] V. Zinage, N. Harutyunyan, E. Verheyden, F. Y. Hadaegh, and S.-J. Chung, “Contractionppo: Certified reinforcement learning via differentiable contraction layers,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.19632>
- [12] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [13] H. Tsukamoto and S.-J. Chung, “Learning-based robust motion planning with guaranteed stability: A contraction theory approach,” *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 6164–6171, 2021.
- [14] R. E. Kalman *et al.*, “Contributions to the theory of optimal control,” *Bol. soc. mat. mexicana*, vol. 5, no. 2, pp. 102–119, 1960.
- [15] T. Cimen, “Survey of state-dependent riccati equation in nonlinear optimal feedback control synthesis,” *Journal of Guidance, Control, and Dynamics*, vol. 35, no. 4, pp. 1025–1047, 2012.
- [16] H. Tsukamoto, S.-J. Chung, and J.-J. E. Slotine, “Contraction theory for nonlinear stability analysis and learning-based control: A tutorial overview,” *Annual Reviews in Control*, vol. 52, pp. 135–169, 2021.
- [17] M. L. Puterman, *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.
- [18] C. Tang, B. Abbatematteo, J. Hu, R. Chandra, R. Martín-Martín, and P. Stone, “Deep reinforcement learning for robotics: A survey of real-world successes,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 27, 2025, pp. 28 694–28 698.
- [19] Z. Ahmed, N. Le Roux, M. Norouzi, and D. Schuurmans, “Understanding the impact of entropy on policy optimization,” in *International conference on machine learning*. PMLR, 2019, pp. 151–160.
- [20] L. Engstrom, A. Ilyas, S. Santurkar, D. Tsipras, F. Janoos, L. Rudolph, and A. Madry, “Implementation matters in deep rl: A case study on ppo and trpo,” in *International conference on learning representations*, 2019.
- [21] M. Andrychowicz, A. Raichuk, P. Stańczyk, M. Orsini, S. Girgin, R. Marinier, L. Hussenot, M. Geist, O. Pietquin, M. Michalski *et al.*,

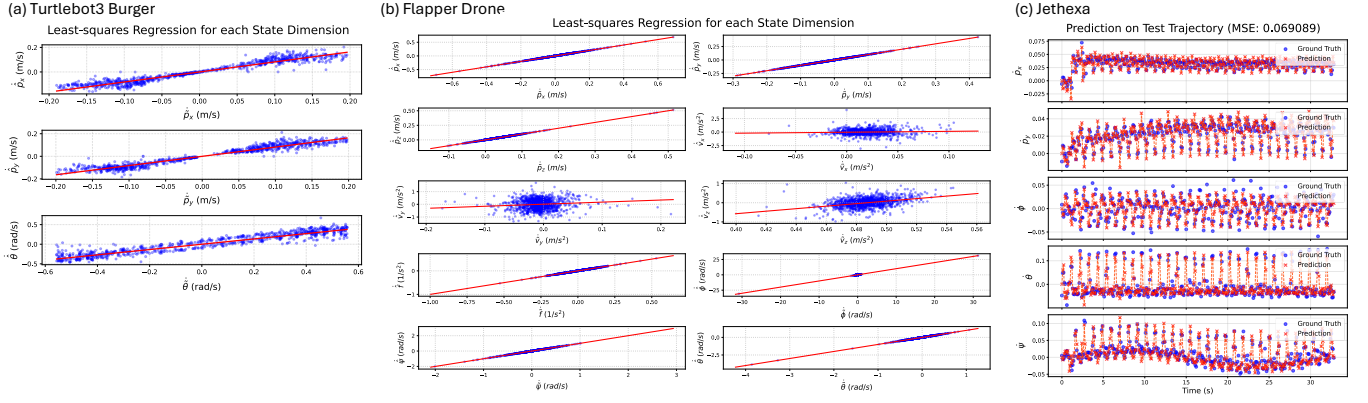


Fig. 16: We plot the results of system identification results for the TurtleBot3 Burger, Flapper Drone, and Jethexa. The identified empirical coefficients for the TurtleBot3 Burger and the Flapper Drone are listed in Table II, and the identified dynamics model ($\mathcal{J}_{\varpi}$) of Jethexa is used in Equation (44).

“What matters for on-policy deep actor-critic methods? a large-scale study,” in *International conference on learning representations*, 2021.

- [22] S. Huang, R. F. J. Dossa, A. Raffin, A. Kanervisto, and W. Wang, “The 37 implementation details of proximal policy optimization,” in *ICLR Blog Track*, 2022, <https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/>. [Online]. Available: <https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/>
- [23] S. Jang, S. Yang, and C. Choi, “Stochastic policies, deterministic minds: A calibrated evaluation protocol and diagnostics for deep reinforcement learning,” *International Journal of Advanced Computer Science & Applications*, vol. 16, no. 10, 2025.
- [24] D. Bertsekas and S. E. Shreve, *Stochastic optimal control: the discrete-time case*. Athena Scientific, 1996, vol. 5.
- [25] A. Montenegro, M. Mussi, A. M. Metelli, and M. Papini, “Learning optimal deterministic policies with stochastic policy gradients,” *arXiv preprint arXiv:2405.02235*, 2024.
- [26] S. Wei, M. Ornik, and H. Tsukamoto, “Conformal contraction for robust nonlinear control with distribution-free uncertainty quantification,” in *IEEE Conference on Decision and Control*, 2025, pp. 3502–3507.
- [27] T.-W. Hsu and H. Tsukamoto, “Statistical guarantees in data-driven nonlinear control: Conformal robustness for stability and safety,” *IEEE Control Systems Letters*, vol. 9, pp. 997–1002, 2025.
- [28] A. Liu, G. Shi, S.-J. Chung, A. Anandkumar, and Y. Yue, “Robust regression for safe exploration in control,” in *Learning for Dynamics and Control*. PMLR, 2020, pp. 608–619.
- [29] S. Singh, S. M. Richards, V. Sindhvani, J.-J. E. Slotine, and M. Pavone, “Learning stabilizable nonlinear dynamics with contraction-based regularization,” *The International Journal of Robotics Research*, vol. 40, no. 10-11, pp. 1123–1150, 2021.
- [30] S. Singh, B. Landry, A. Majumdar, J.-J. Slotine, and M. Pavone, “Robust feedback motion planning via contraction theory,” *The International Journal of Robotics Research*, vol. 42, no. 9, pp. 655–688, 2023.
- [31] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2017, pp. 23–30.
- [32] E. Smirnova, E. Dohmatob, and J. Mary, “Distributionally robust reinforcement learning,” *arXiv preprint arXiv:1902.08708*, 2019.
- [33] H. L. Royden and P. Fitzpatrick, *Real analysis*. Macmillan New York, 1988, vol. 32.